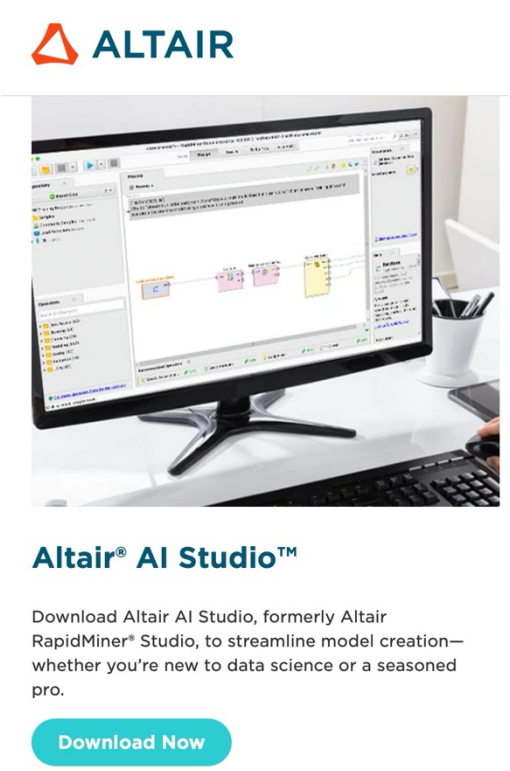
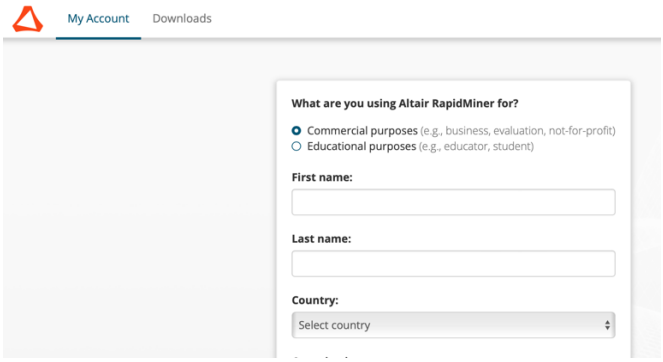
# Altair AI Studio (RapidMiner) 安裝

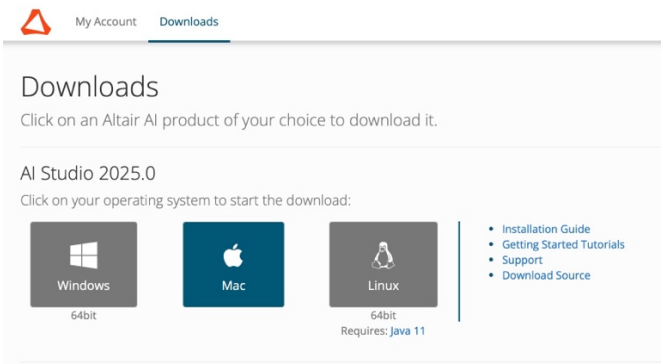
國立清華大學 學習科學與科技研究所 區國良

# 下載及安裝

下載網址
https://altair.com/altair-rapidminer-free-trials



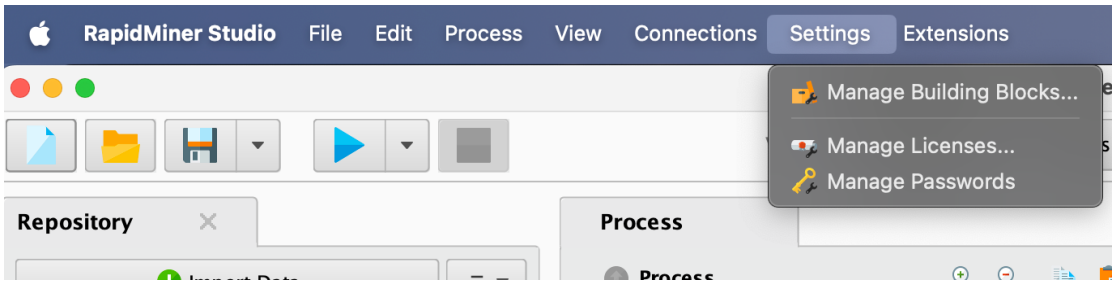




註冊時請以結尾為 edu.tw 的信箱註冊，便於日後申請教育版解鎖進階功能，並選擇適合的平台版本，目前支援 Windows, Mac 以及 Linux，請自行依指示安裝。

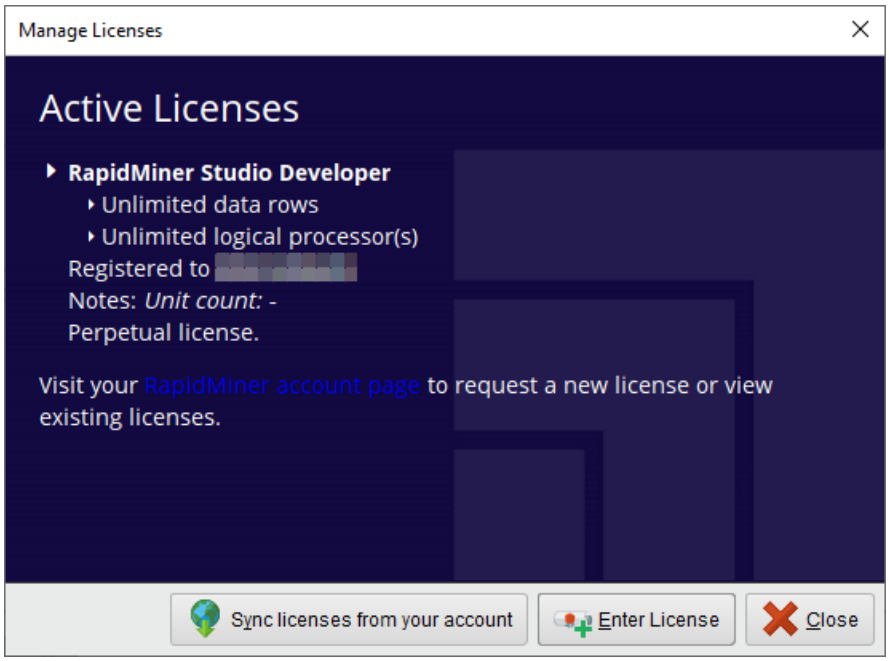
# 啟動授權碼

由於試用版的資料處理能力只限 10000 列，且 AutoModel 服務會有若干的限制，雖然沒有使用期限，但實務處理大量數據時常有捉襟見肘之憾。RapidMiner 其實提供了教育版的授權申請，雖然一段時間（一年左右）就要上線更新授權碼，但其授權後就很大方地解鎖許多進階功能，尤其提供了所有 AutoModel 的服務，處理資料量也無上限，建議同學不妨在安裝好 RapidMiner 後優先申請為教育版。申請時首先要在 RapidMiner 網站註冊帳號（請自行至官網 https://rapidminer.com/ 申請，接著步驟如下：

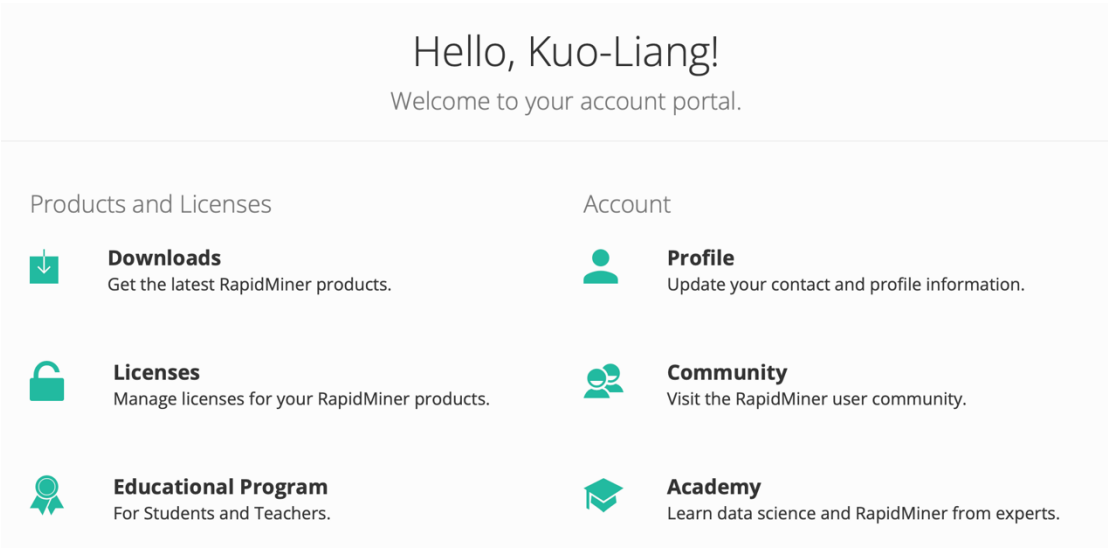
Step 1: 在 Settings 中點選 Manage Licenses




Step 2: 點選下方 Visit your RapidMiner account page




Step 3: 進入左下角的 Educational Program，並按照說明申請

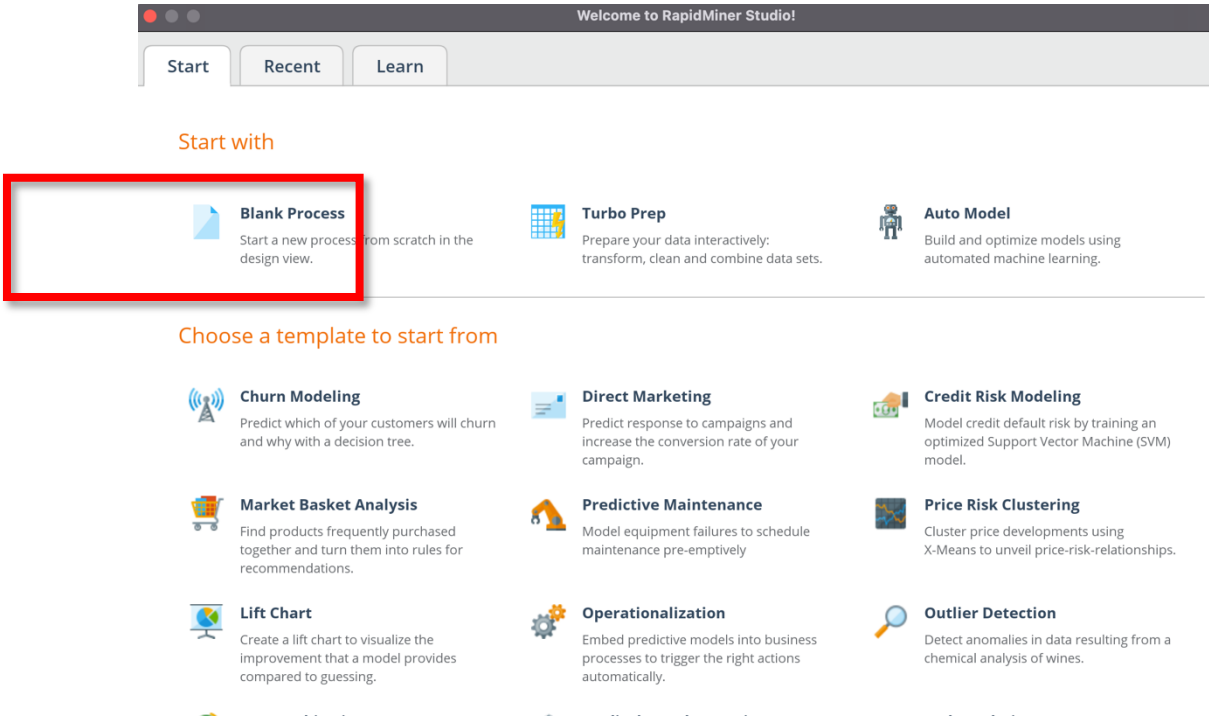



Step 4: 申請成功後，在 Licenses 頁面中，即可見到可用的授權了

Step5: 最後一步，回到 RapidMiner App 的 Manage Licenses，下方即可將 App 與官網的授權碼同步了

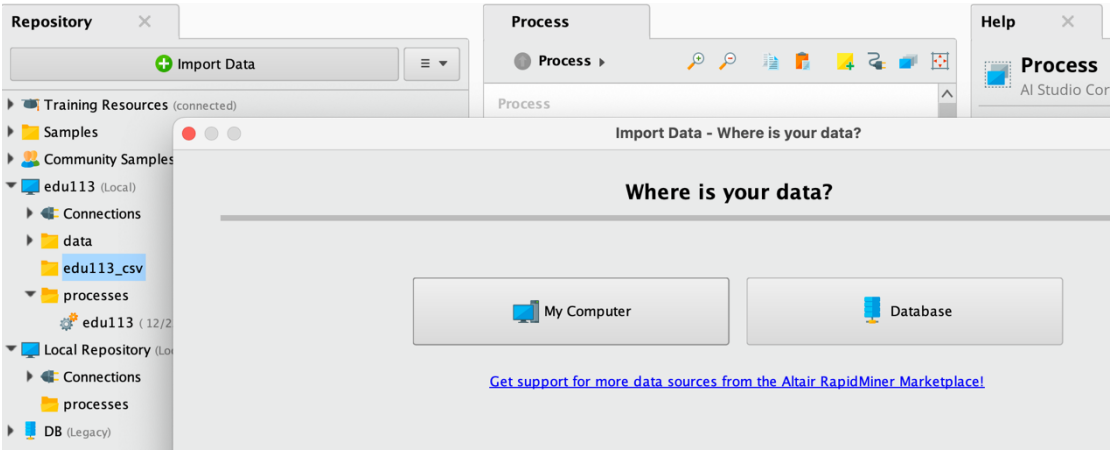
# 啟動

開啟 RapidMiner 後，在歡迎視窗中的 Start With 提供了三種使用的環境，包含 Blank Process、Trubo Prep 以及 Auto Model，其中 Turbo Prep 提供了機器學習過程中最花時間的「洗資料」服務，Auto Model 則提供了大量「傻瓜型」的快速分析工具，讓使用者一次享受多種演算法及參數調整、評估等工作；對於初學者，我還是建議由全新的空白操作開始，除了能掌握各項工具的特色外，更能體驗機器學習的完整流程。現在，就請選擇 Blank Process，讓我們開始一個全新空白的操作流程吧！



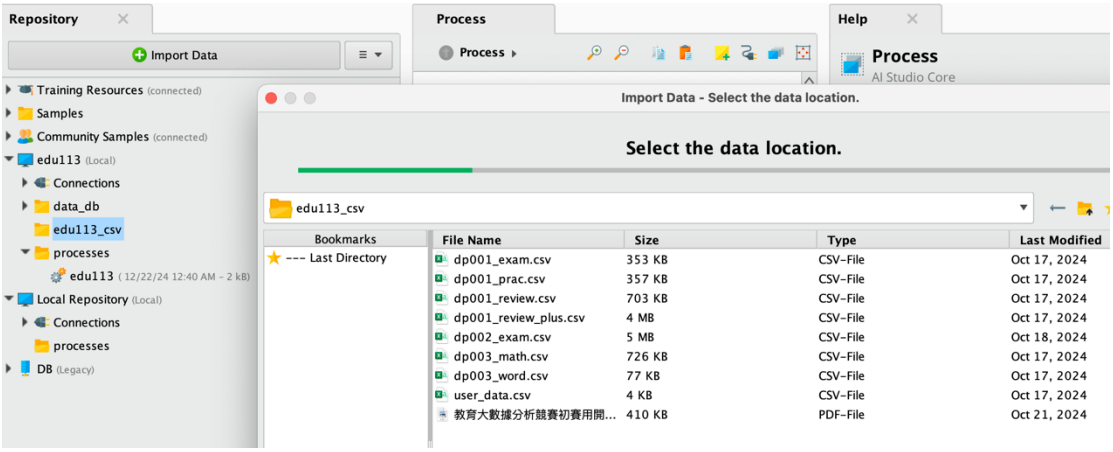

# 匯入資料表

資料表的來源支援本機檔案，如 csv 等格式，另外也遠端資料庫即時查詢



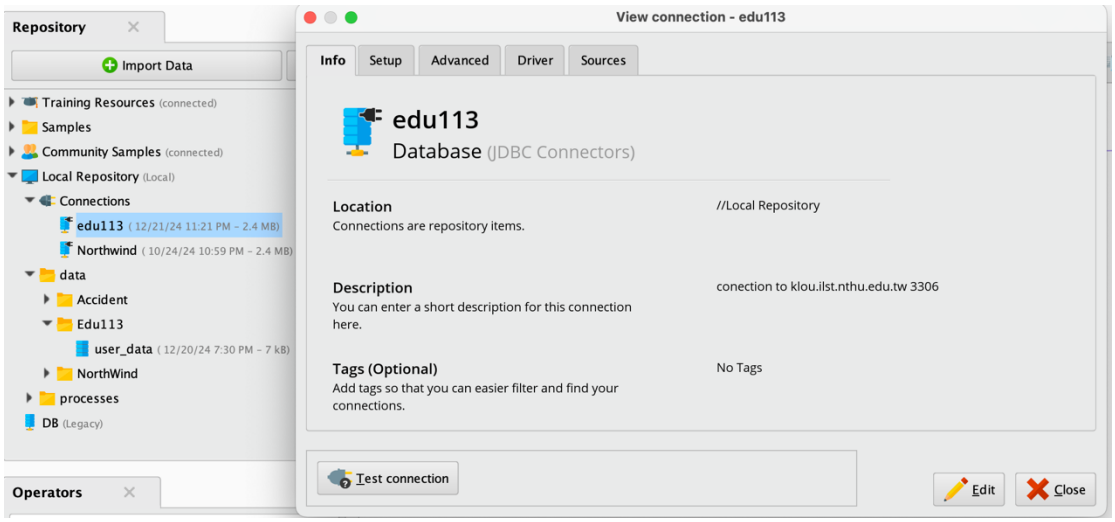
# 本地檔案

在 Repository 頁簽中點選 Import Data，依提示匯入指定資料表，支援 excel 及 csv 格式

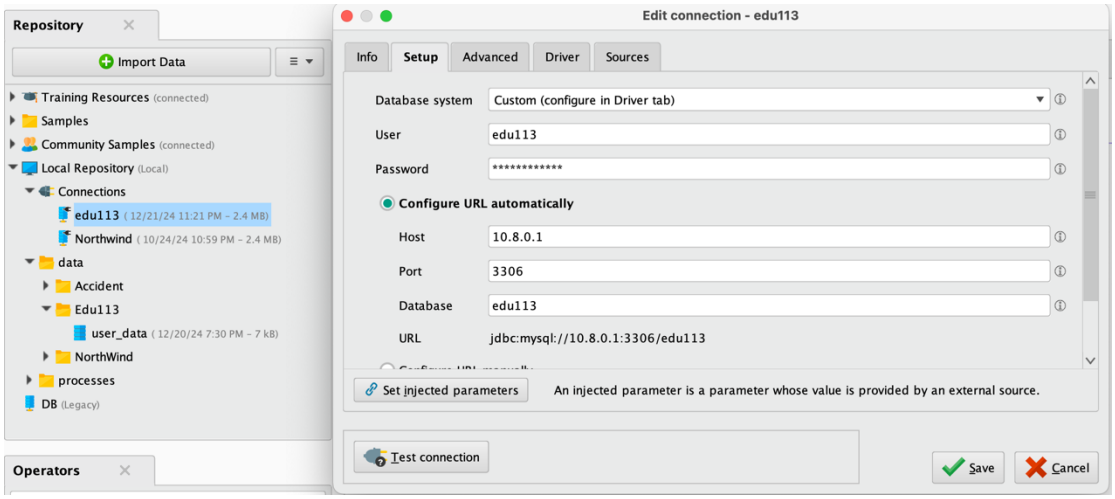


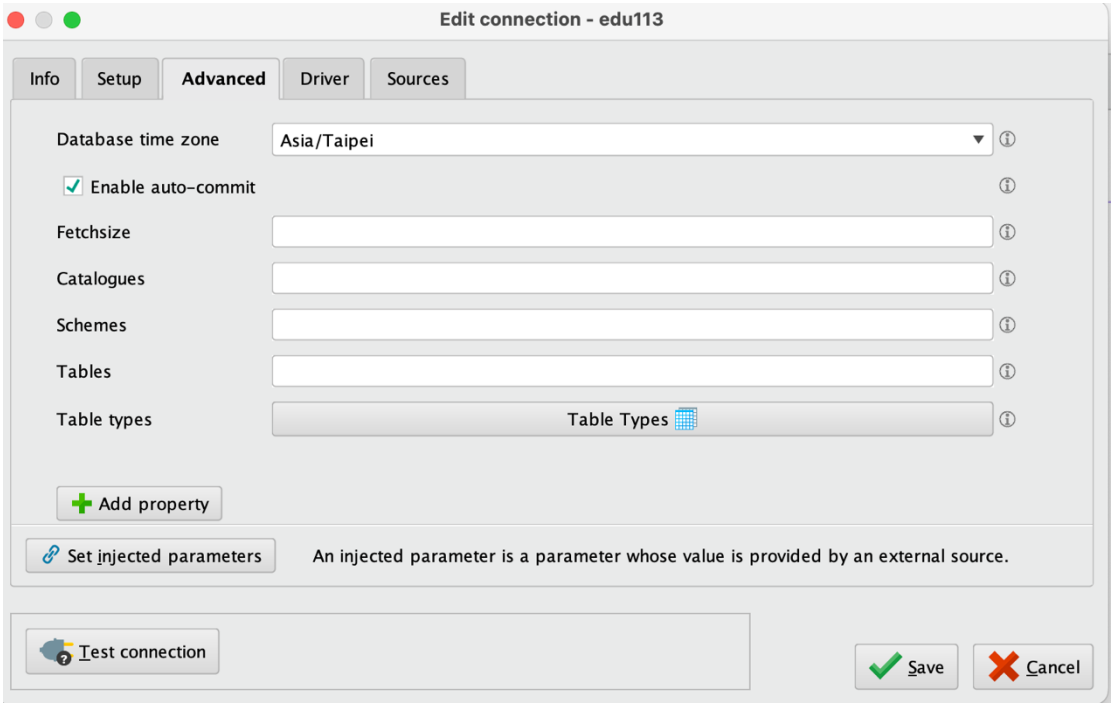

# 遠端資料庫

1. 建立資料庫連結 connections



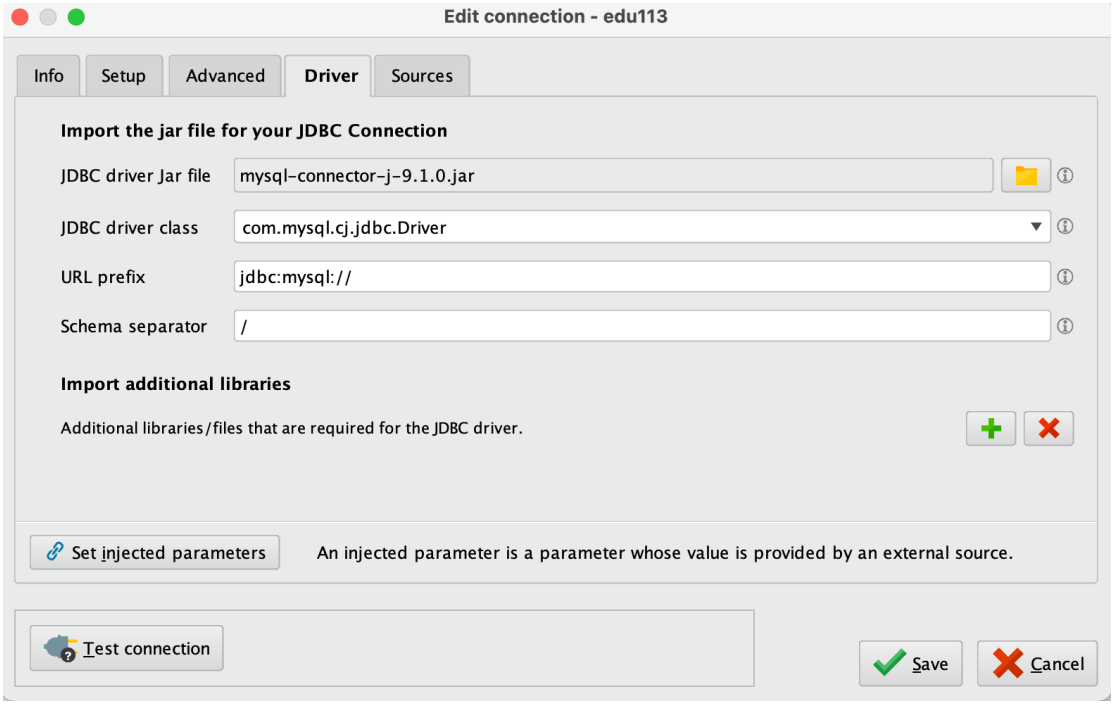

2. 修改時區為 Asia/Taipei





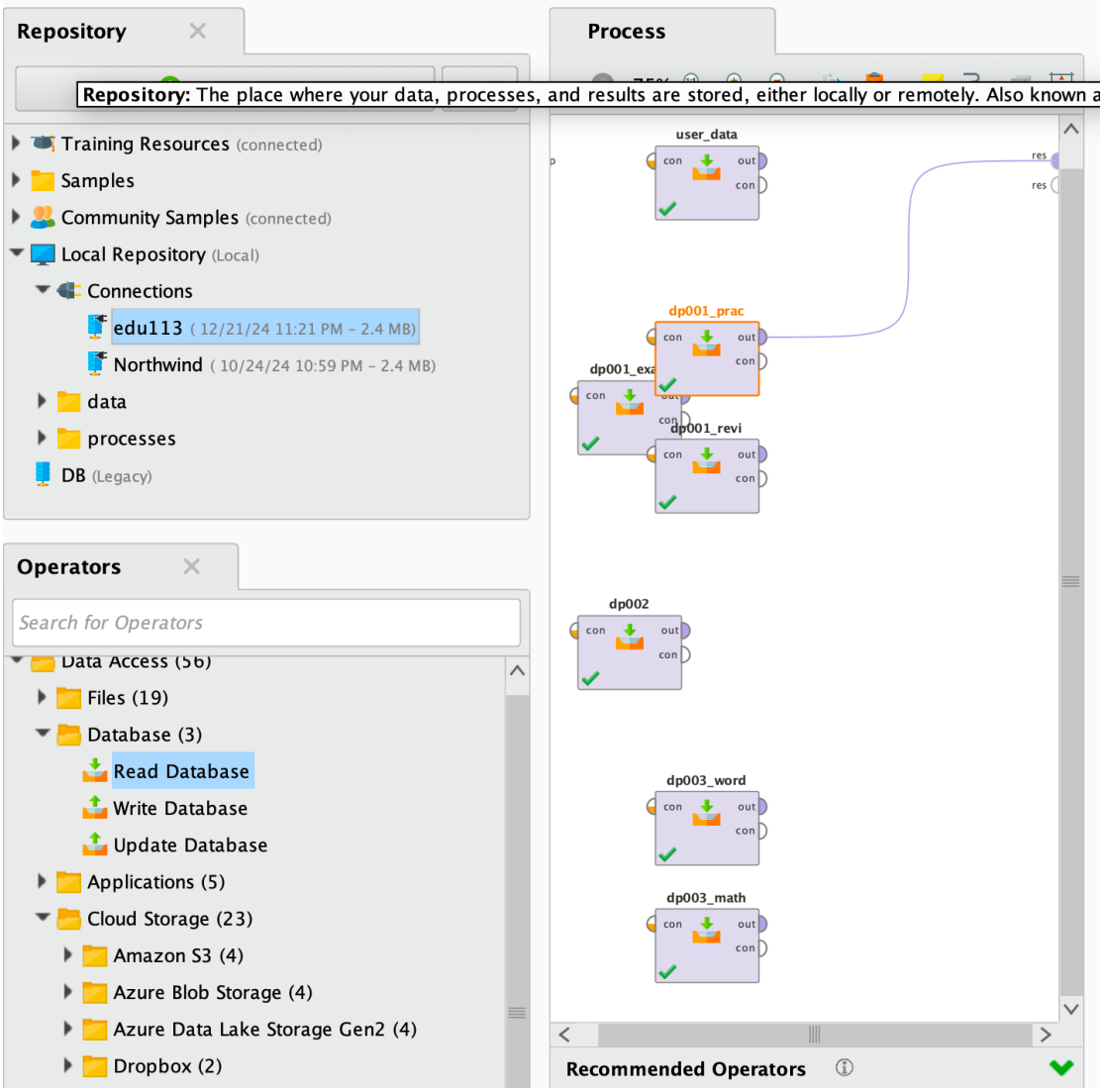

3. 安裝 JDBC driver

請至 https://dev.mysql.com/downloads/connector/j/ 依提示下載後，回到 RapidMiner 繼續安裝

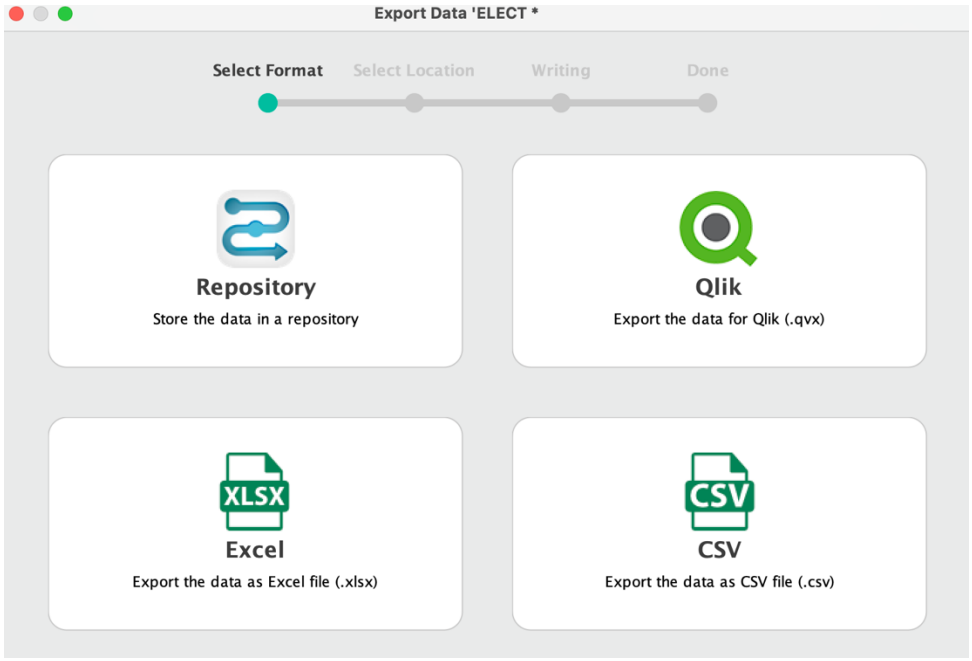


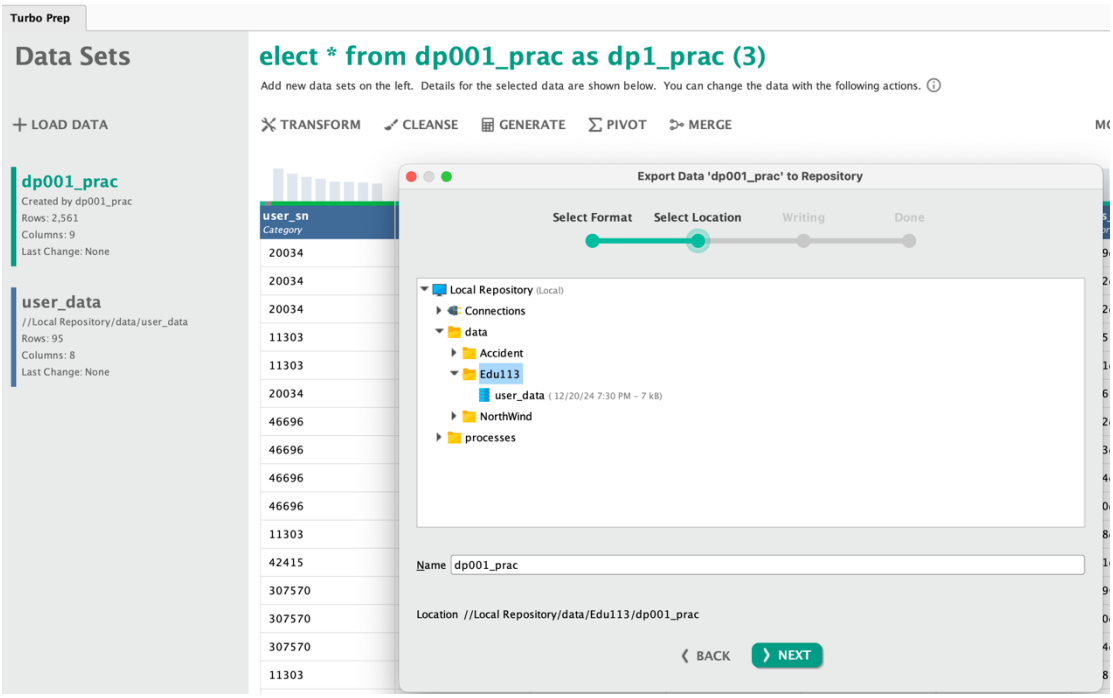

4. 讀取資料表

使用 Read Database 設定 SQL 分別讀取各別的表格或指定的查詢資料集

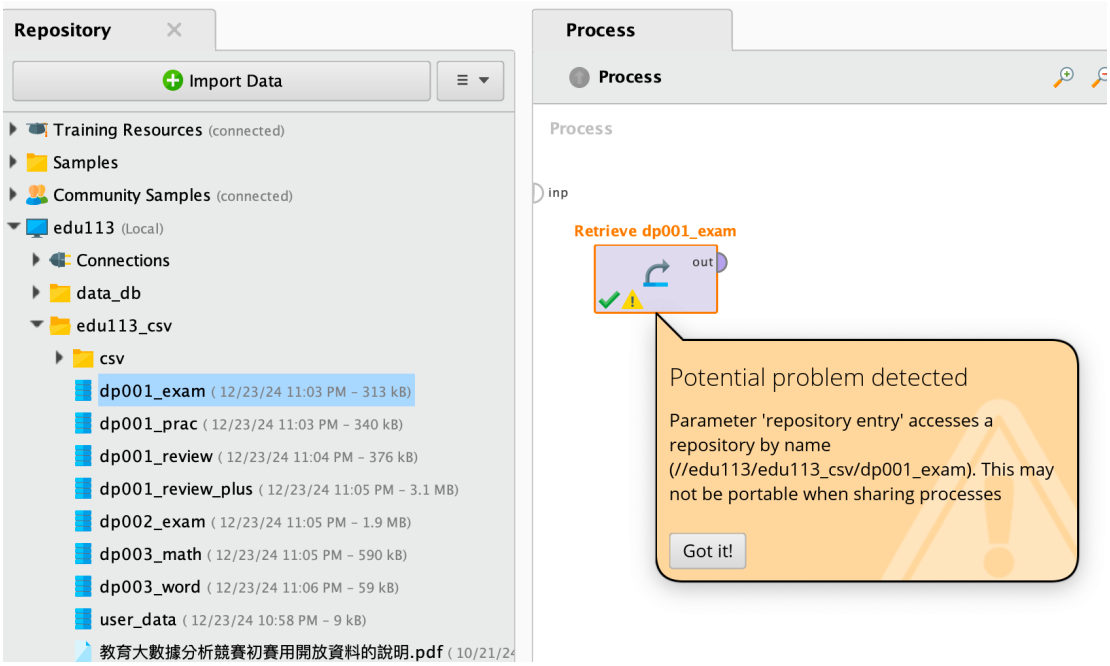



5. 將查詢的結果（Result）頁簽中的 Turbo Prep 功能，將結果設按右鍵，設定 Export 以及 Repository 以方便後續取用



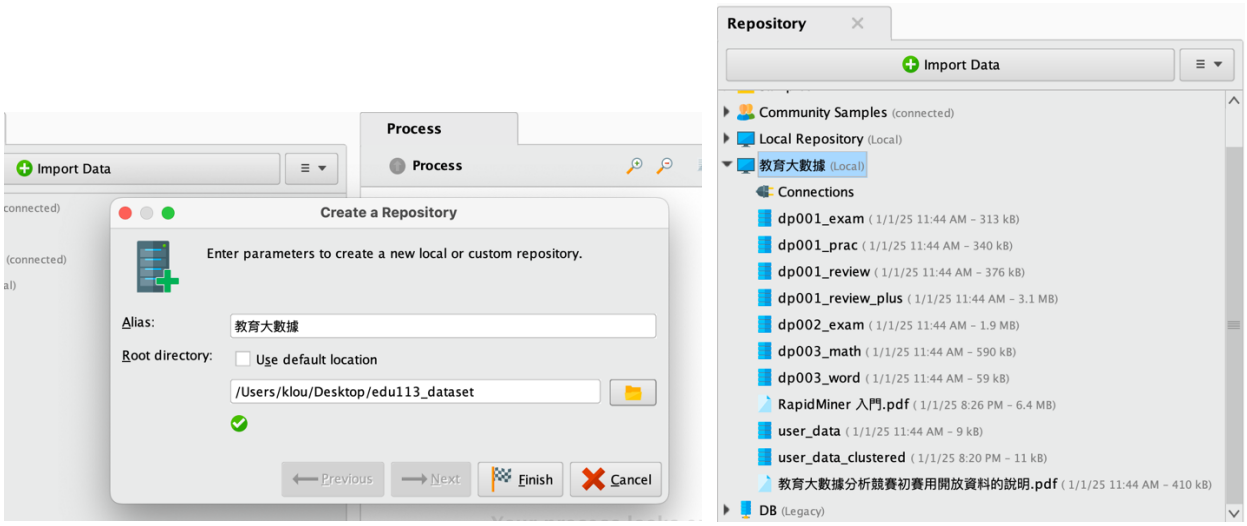



若以教育大數據 2024 競賽所公開的資料集匯入完成後的畫面大致如下，警告訊息說明未來在分享 process 時可能會出現的情況，使用者需留意檔案名稱及路徑的一致性。

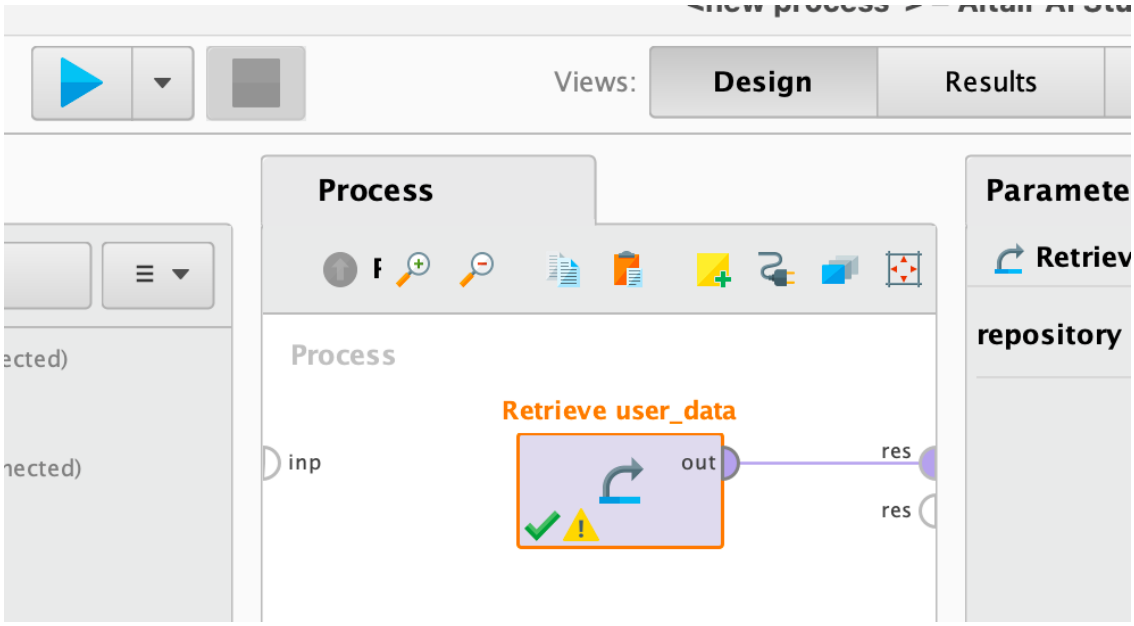


# 資料探索

可以 GitHub 下載教學用資料集(修改過的版本)
https://github.com/KuoLiang/r-klou/tree/main/15-EduBigDataGame
將目錄中的所有檔案暫置於電腦桌面上，再由 RapidMiner Import Data 右方的向下箭頭新增資料儲存庫 (Create a Respository)，移除 Use default location 並修改路徑如左下

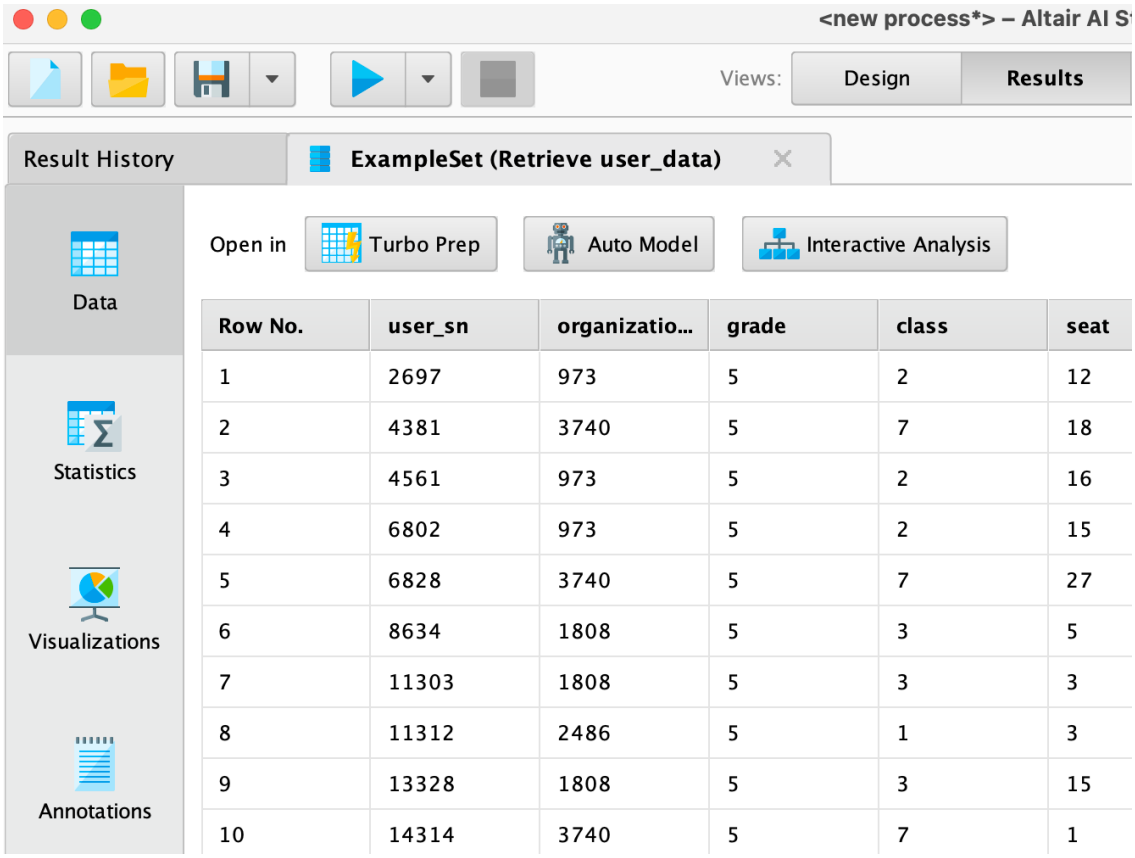



右上圖為匯入資料集後的畫面，接下來將資料集 user_data 置入 process 頁簽，其 out (output) 引出至 res (result)，按下藍色執行鈕：



得到如下 results 顯示，其中左方共有 Data, Statistics, Visualizations, and Annotations 選項:

1. Data

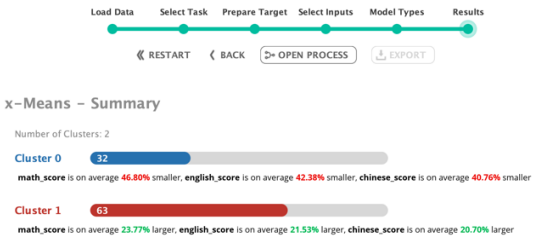



| Row No. | user_sn | organizatio... | grade | class | seat |
|---|---|---|---|---|---|
| 1 | 2697 | 973 | 5 | 2 | 12 |
| 2 | 4381 | 3740 | 5 | 7 | 18 |
| 3 | 4561 | 973 | 5 | 2 | 16 |
| 4 | 6802 | 973 | 5 | 2 | 15 |
| 5 | 6828 | 3740 | 5 | 7 | 27 |
| 6 | 8634 | 1808 | 5 | 3 | 5 |
| 7 | 11303 | 1808 | 5 | 3 | 3 |
| 8 | 11312 | 2486 | 5 | 1 | 3 |
| 9 | 13328 | 1808 | 5 | 3 | 15 |
| 10 | 14314 | 3740 | 5 | 7 | 1 |

1.1 Turbo Prep 清洗資料

1.2 Auto Model 自動建模

本資料集可嘗試著數學、國語及英語分數進行 Clusters 分析，目前提供 K-Means 以及 X-Means 兩種演算法，下圖為 X-Means 所建構出來的兩群，其中 clouster0 可解釋為低分數群，Clouster1 可解釋為高分數群

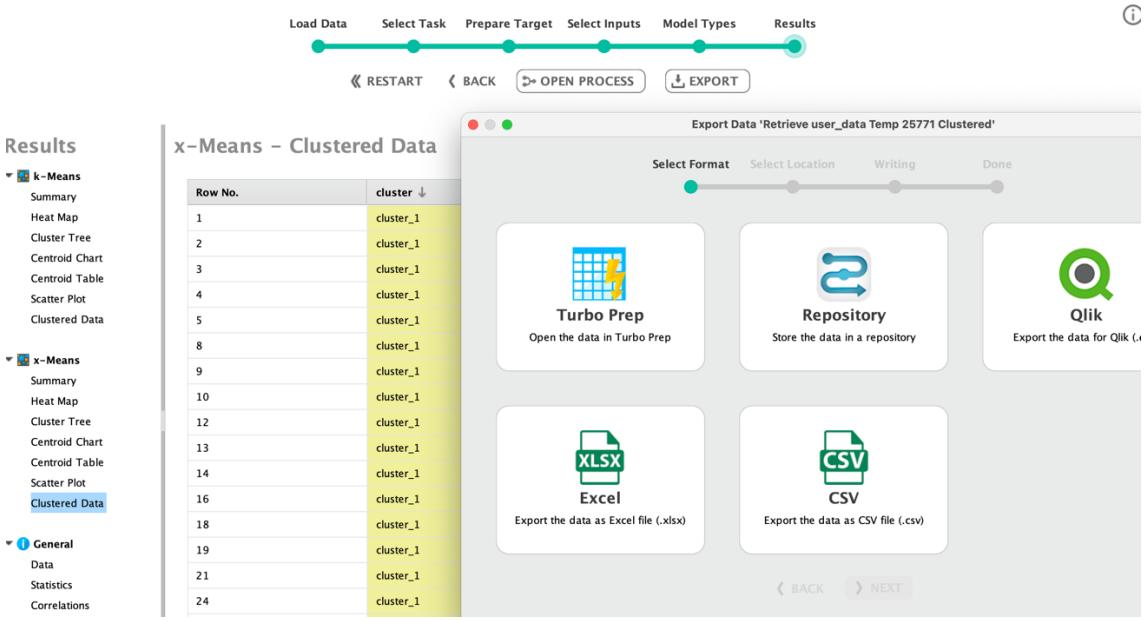



Summary

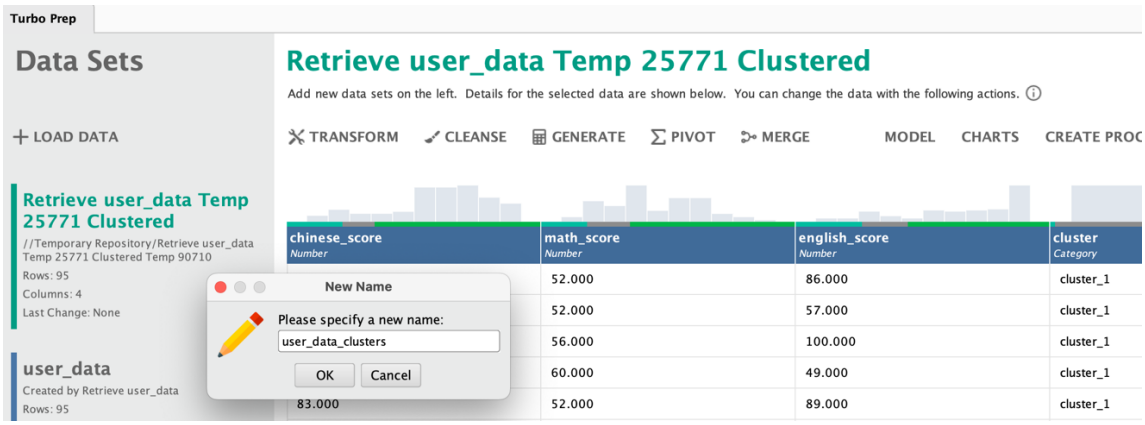



Scatter Plot

可進一步將分群的結果利用 Export 功能中的 Turbo Prep 作為新的欄位




可重新命名為新的資料集 (Data Set) ，並以資料集之 Merge 功能合併 (join) 原始表格，若無可用於交集的 key 值，可點選 “use row numbers as keys”，預覽無誤後，執行 merge

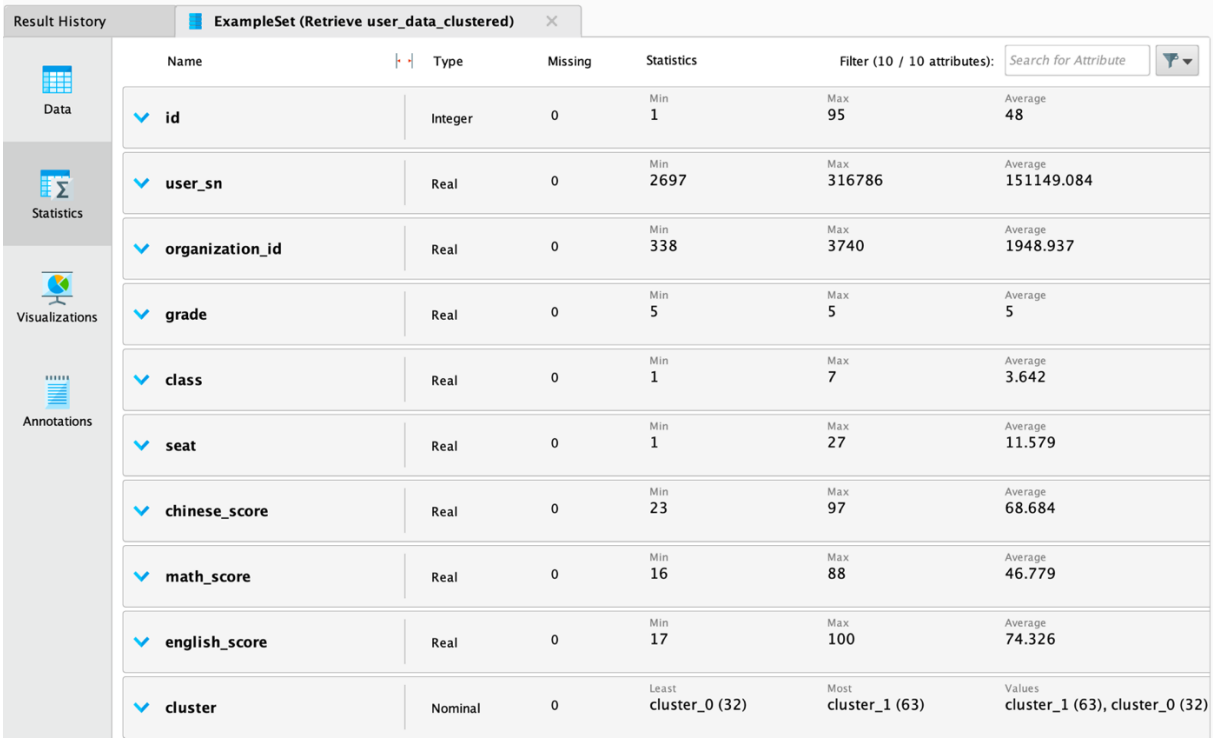



或將該資料集按右鍵後 export 到 repository 供日後使用

## 1.3 Interactive Analysis 互動分析

目前教育版不支援

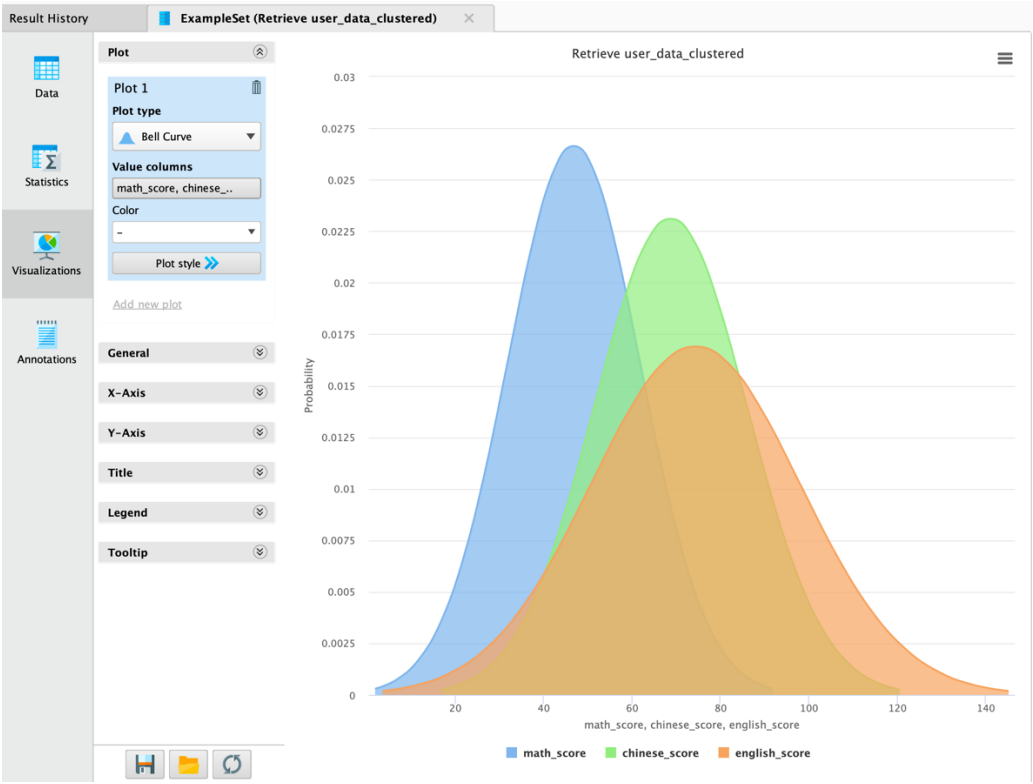
2. Statistics




3. Visualizations

將分群後的結果以視覺化方式檢視






4. Annotations

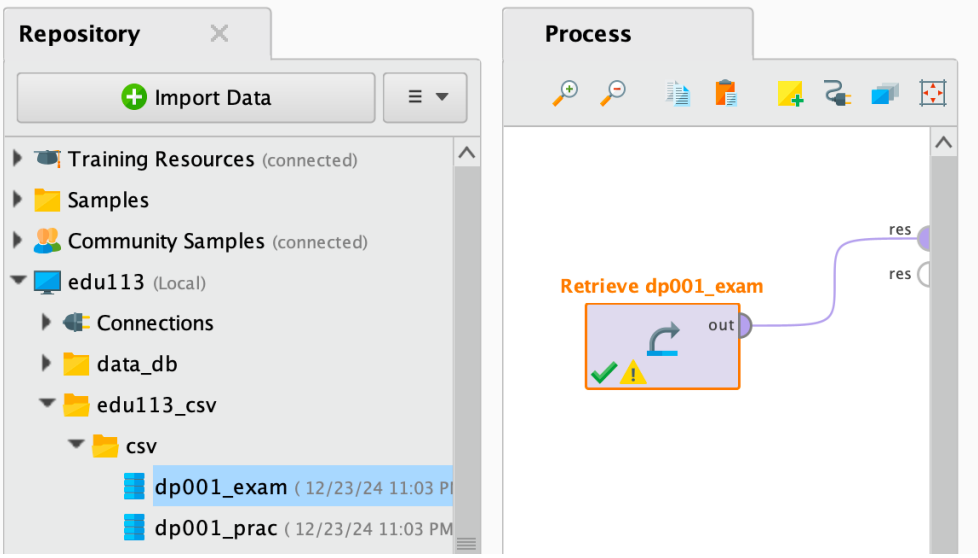
Nothing special ，這裏就只是系統的回應及註記罷了

# 資料探索: dp001_exam

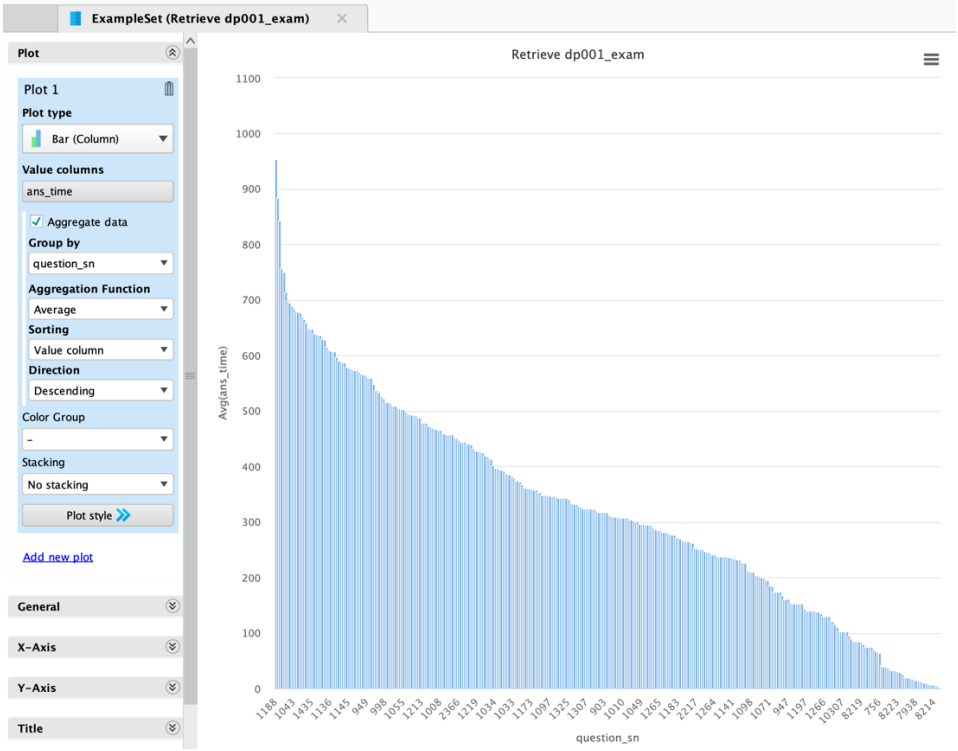
使用者在學習平台 dp001 影片檢核點作答資料，可能探索的方向為 (1) 某些題目特別的艱難，若以題目為中心進行分析時，可能得到某些題目平均作答時間比較長；(2) 使用者沒有注意觀看影片，遇到檢核點時仍在處理別的工作，若以使用者為中心進行分析時，可能得到某些使用者平均作答時間拉長

(1) 以題目為中心進行分析

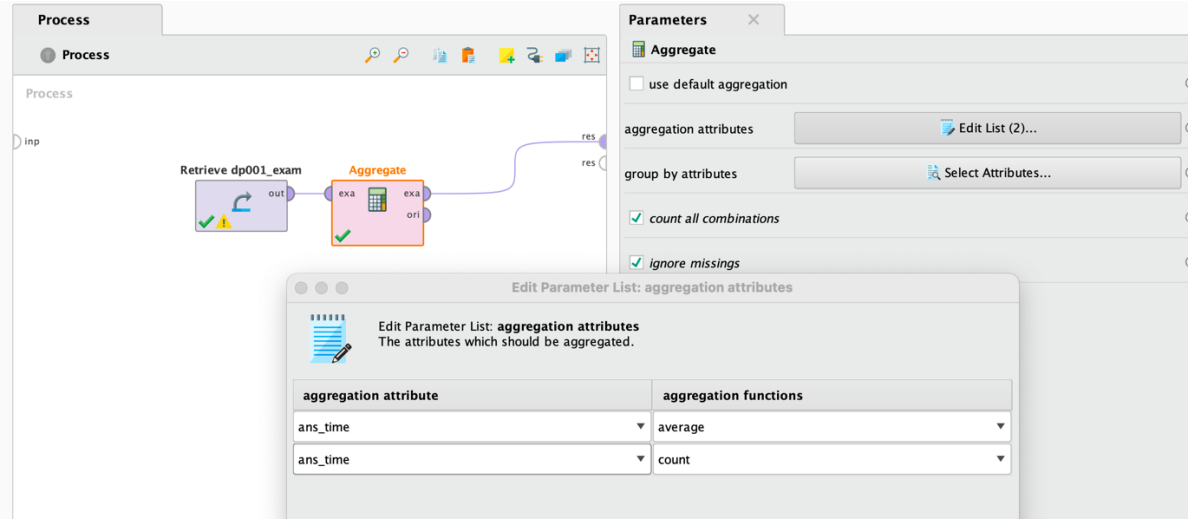
取得 dp001_exam 並將 out 連接到 res



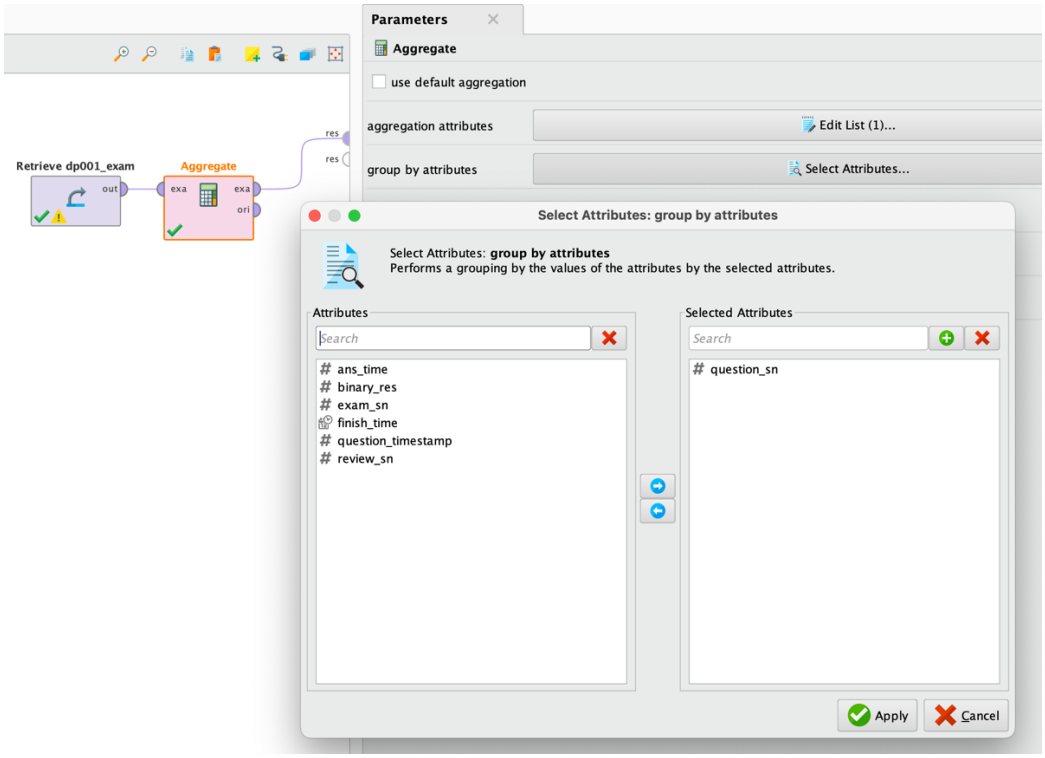

在 Visualizations 選擇 bar 圖示，依下圖調整各參數，得出結果可見依平均作答時間排序後的結果



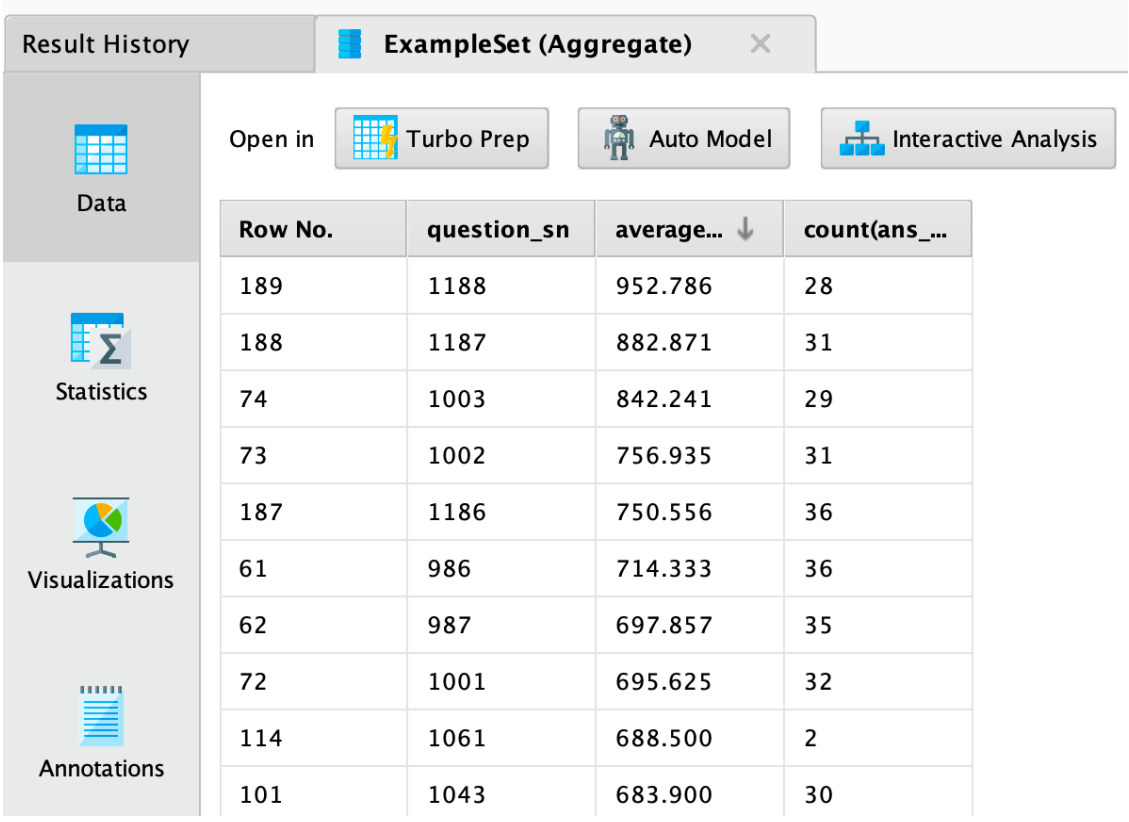

將資料集依上圖的結果製作新的資料表，使用 Aggregate operator，在 aggregate attributes 中選擇 ans_time 分別進行 average 以及 count 運算




在 group by 則依 question_sn 進行群組

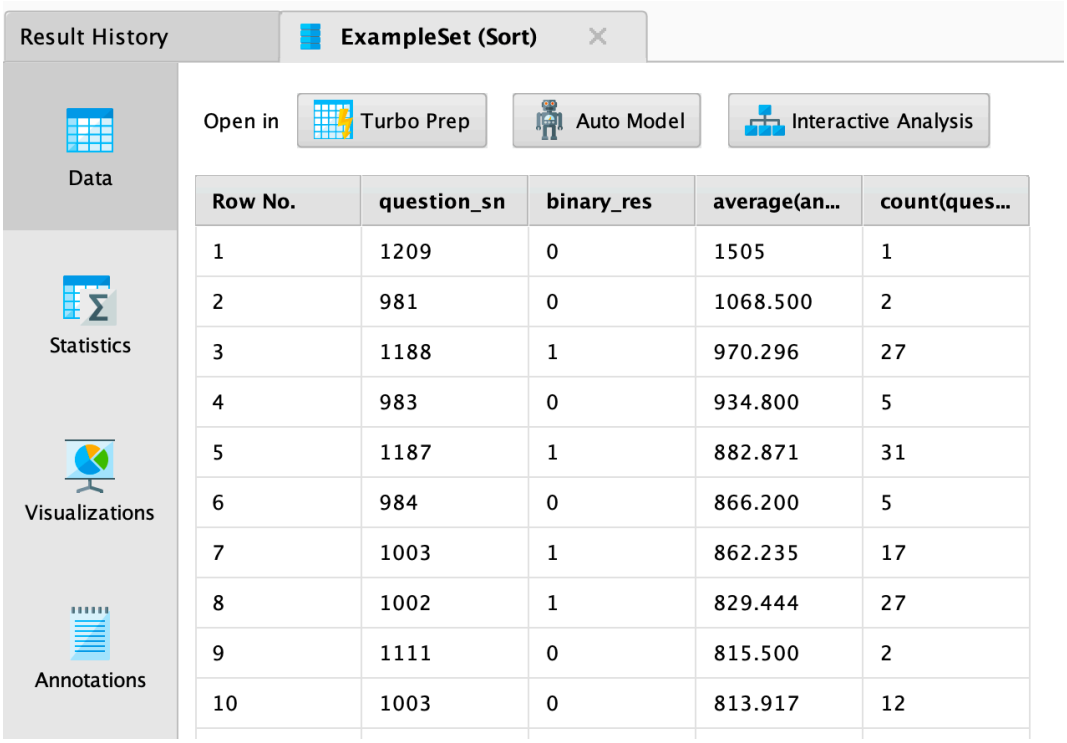



得到共 328 列，最高為 question_sn = 1188 , 該問題在 28 次的回答中平均花費 952.786 秒



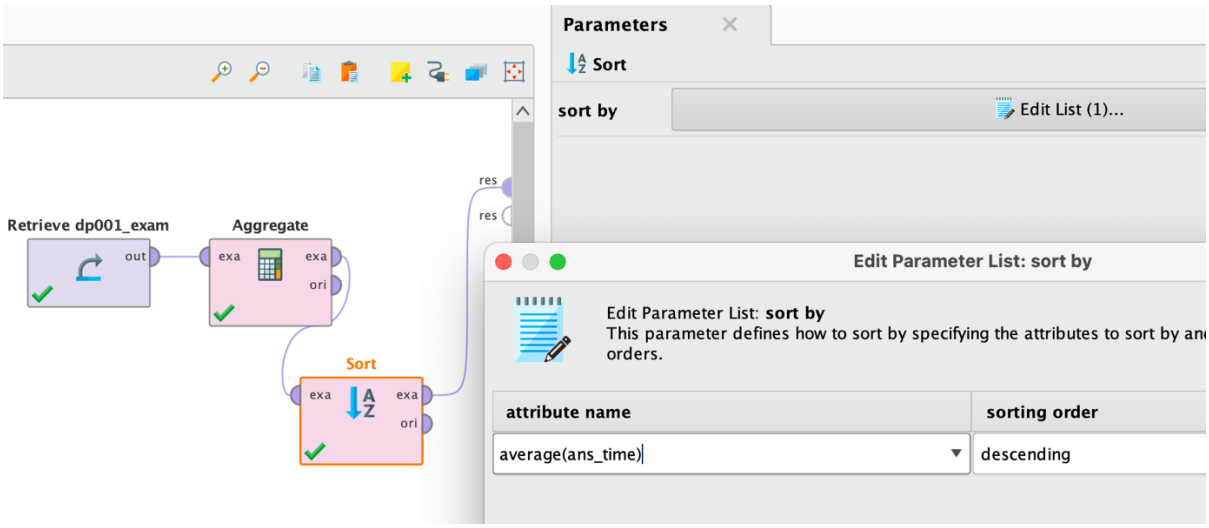

| Row No. | question_sn | average... | count(ans_... |
|---|---|---|---|
| 189 | 1188 | 952.786 | 28 |
| 188 | 1187 | 882.871 | 31 |
| 74 | 1003 | 842.241 | 29 |
| 73 | 1002 | 756.935 | 31 |
| 187 | 1186 | 750.556 | 36 |
| 61 | 986 | 714.333 | 36 |
| 62 | 987 | 697.857 | 35 |
| 72 | 1001 | 695.625 | 32 |
| 114 | 1061 | 688.500 | 2 |
| 101 | 1043 | 683.900 | 30 |

若 Aggregate operator 中增加 binary_res 將答對與答錯的題目及時間分開統計，則得到以下線果，其中 question_sn = 1188 共 28 次的作答中，有 27 次是正確的，且平均花費 970 秒左右：




| Row No. | question_sn | binary_res | average(an... | count(ques... |
|---|---|---|---|---|
| 1 | 1209 | 0 | 1505 | 1 |
| 2 | 981 | 0 | 1068.500 | 2 |
| 3 | 1188 | 1 | 970.296 | 27 |
| 4 | 983 | 0 | 934.800 | 5 |
| 5 | 1187 | 1 | 882.871 | 31 |
| 6 | 984 | 0 | 866.200 | 5 |
| 7 | 1003 | 1 | 862.235 | 17 |
| 8 | 1002 | 1 | 829.444 | 27 |
| 9 | 1111 | 0 | 815.500 | 2 |
| 10 | 1003 | 0 | 813.917 | 12 |

想再進一步了解這 28 次回答是否都是由同一人所為，則需要關聯到 dp001_review 表格，這稍後再說明，目前可以先再針對現有單一表格的資料依不同的欄位值再做進一步分群查詢，例如再依答對答錯 (binary-res) 的結果切分，方式是在 Aggregate operator 中增加 group by attributes ，並加上 Sort operator 調整排序如下：

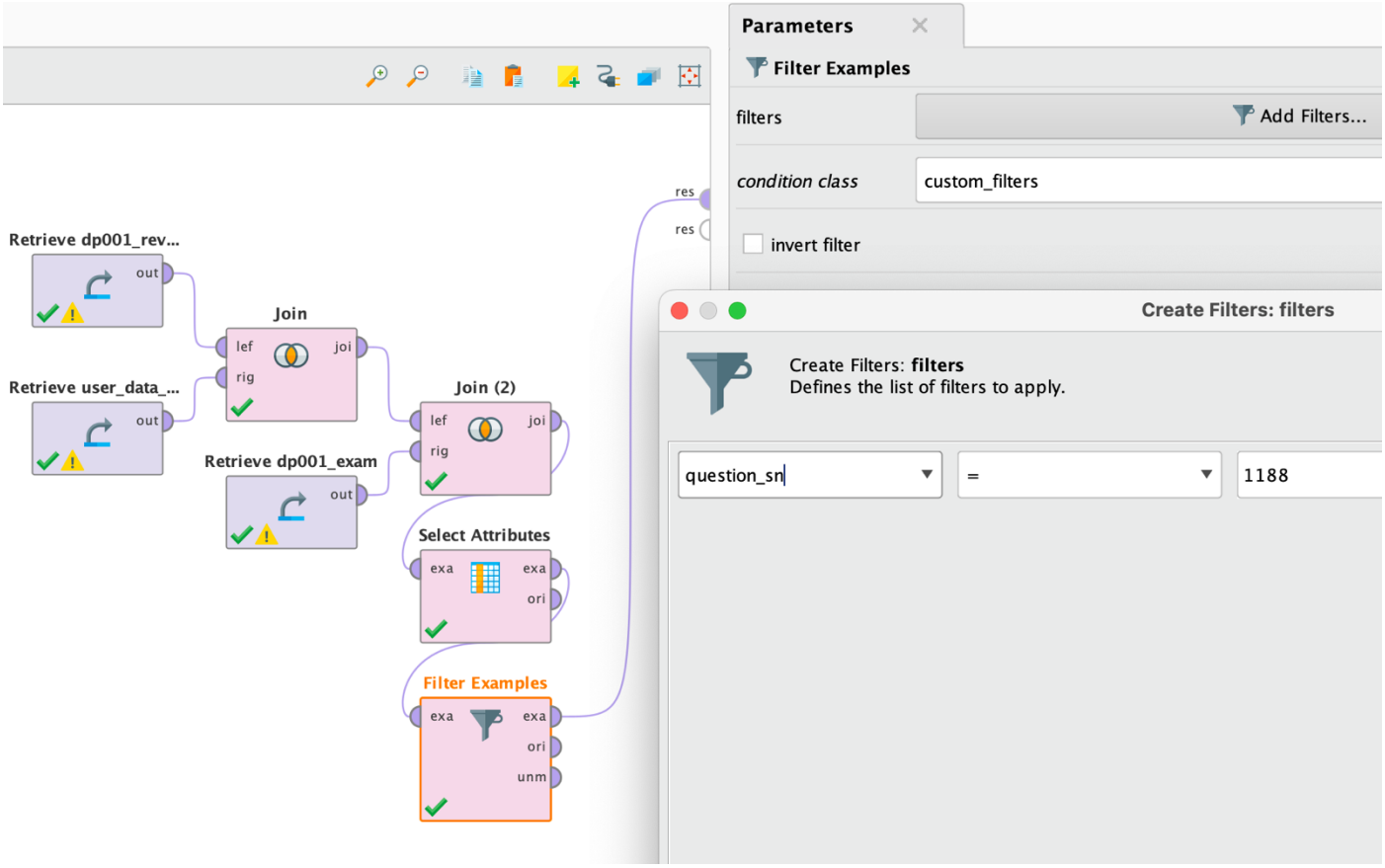



如此，考慮不同題目及作答正確性之後，答題平均秒數最久的前十名中，第一名為 question_sn = 1209 大約花了 25 分鐘 （1505 秒）進行作答，而且還答錯了 (binary_res = 0) ! ; 第二名的檢核點 question_sn = 981 有兩次作答記錄，平均下來也花了超過 17 分鐘作答，結果兩次答案都是錯的。由於影片檢核點的問題通常不會太難，且都和影片內容有關，所以可以猜想這些同學其實跟本沒有在專心看影片；但第三名的 question_sn = 1188 則正好相反，共有 27 次作答記錄，平均花費 16 分鐘以上作答，而且這 27 次答案都是對的，這就有可能真是因為題目太難，或其它目前未知的原因造成，有待進一步詳細分析。

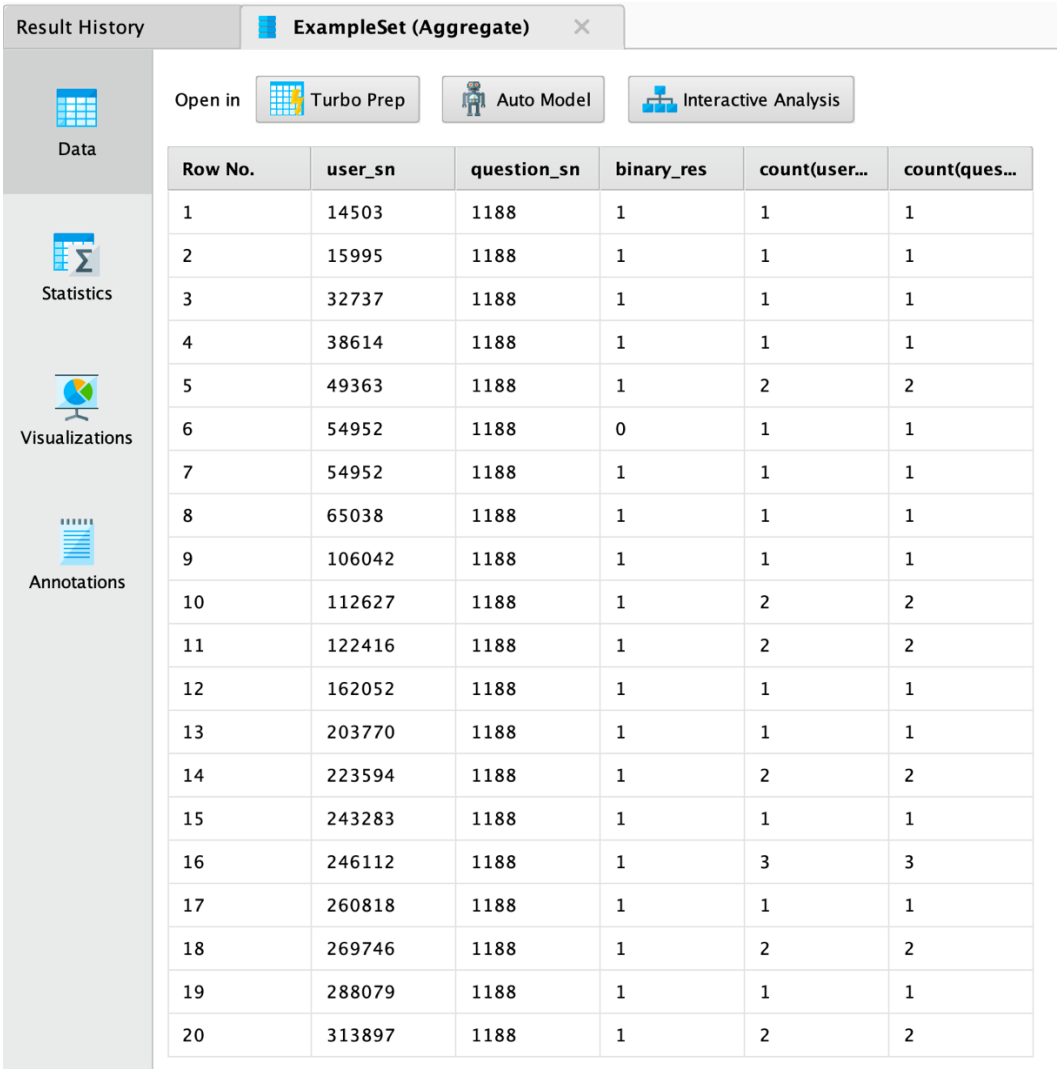
若需將結果輸出為 csv 檔案，可使用 Wirte CSV operator

(2) 以使用者為中心進行分析

首先將 user_data, dp001_review 以及 dp001_exam 「分別」進行 join operator, 由於使用者資料及期末成績記錄在 user_data ，而我們想要的檢核點行為記錄在 dp001_exam ，兩者之間沒有關聯，必須透過 dp001_review 作為中介的 Entity, 故先將 dp001_review 作為 left Entity, user_data 作為 right Entity 進行 left join, 其結果再與 dp001_exam 做 inner join, 得到一張大表，由於其中有許多應該用不到的欄位，例如 class, set 等，可自行利用 select attributes operator 刪除，若再加上 Filter Examples operator，則可針對特定欄位值篩選，例如下圖只列出 question_sn = 1188 的 28 次作答情況，可見大部分皆為 cluster_1 (高分群)的同學，少部分為低分群的同學（只有 3 位同學共 5 次作答），有些同學重複作答，分別有答對和答錯的經驗 (binary_res)，若要再詳細觀察，可再將 user_sn 以 Aggregate operator 作為 group by 的對象

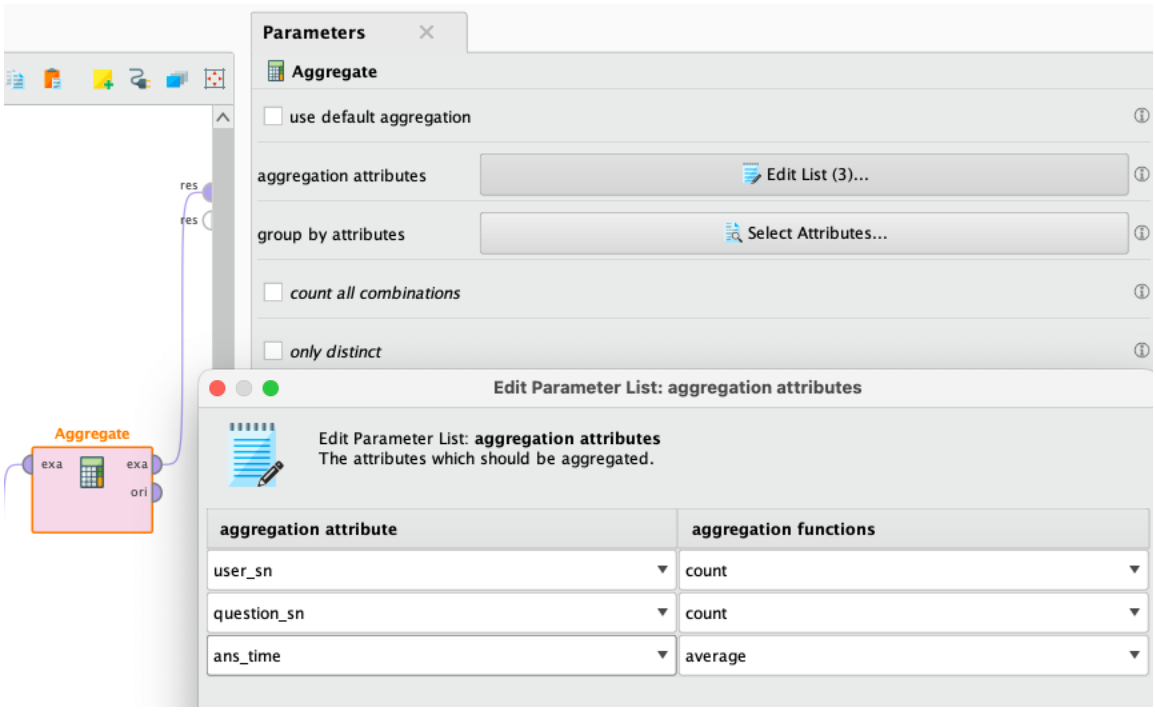


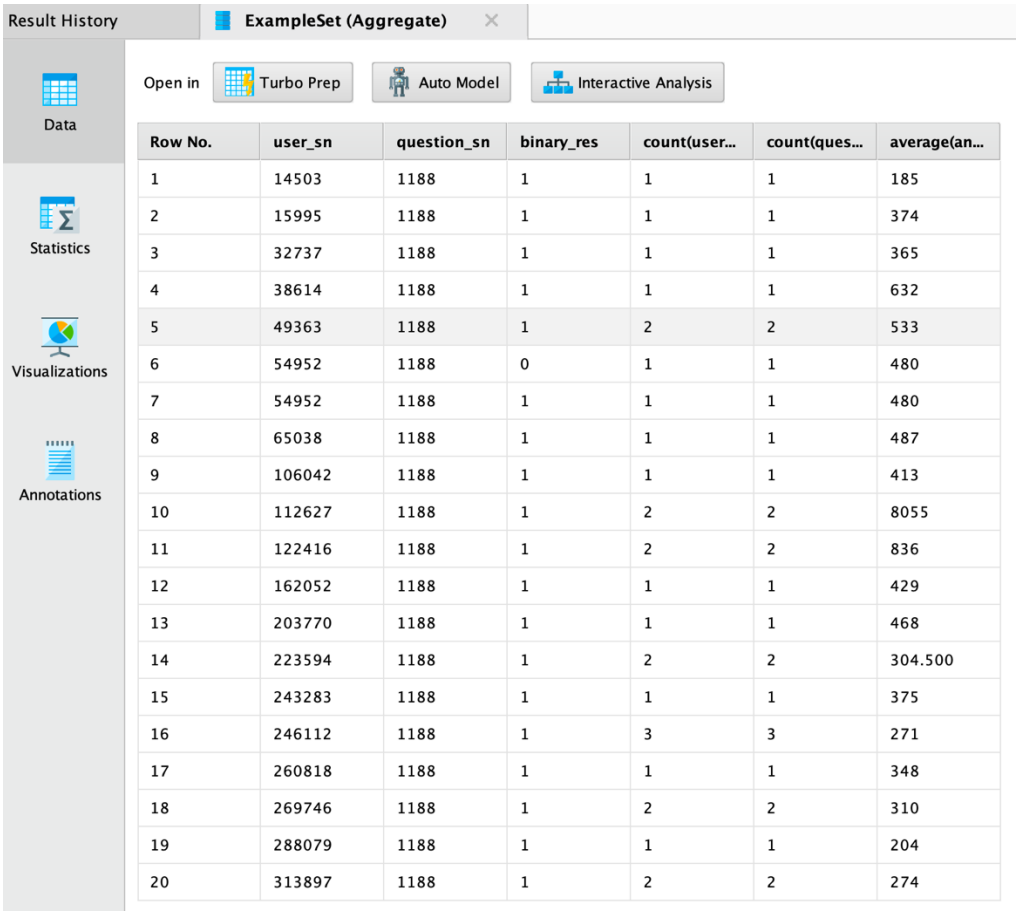

共有 20 筆符合 question_sn = 1188 的作答記錄，其中只有第 6 列，也就是 user_sn = 54952 的同學答錯了，還記得前面曾提到共有 27 位同學曾答對這題，且花費時間較長，目前看來有部分同學重複答題，所以其實是 19 位不同的學生答對本題，另外有一位答錯（user_sn = 54952），在扣附重複答題的記錄後，其實是共 20 位同學曾參與作答。




| Row No. | user_sn | question_sn | binary_res | count(user... | count(ques... |
|---|---|---|---|---|---|
| 1 | 14503 | 1188 | 1 | 1 | 1 |
| 2 | 15995 | 1188 | 1 | 1 | 1 |
| 3 | 32737 | 1188 | 1 | 1 | 1 |
| 4 | 38614 | 1188 | 1 | 1 | 1 |
| 5 | 49363 | 1188 | 1 | 2 | 2 |
| 6 | 54952 | 1188 | 0 | 1 | 1 |
| 7 | 54952 | 1188 | 1 | 1 | 1 |
| 8 | 65038 | 1188 | 1 | 1 | 1 |
| 9 | 106042 | 1188 | 1 | 1 | 1 |
| 10 | 112627 | 1188 | 1 | 2 | 2 |
| 11 | 122416 | 1188 | 1 | 2 | 2 |
| 12 | 162052 | 1188 | 1 | 1 | 1 |
| 13 | 203770 | 1188 | 1 | 1 | 1 |
| 14 | 223594 | 1188 | 1 | 2 | 2 |
| 15 | 243283 | 1188 | 1 | 1 | 1 |
| 16 | 246112 | 1188 | 1 | 3 | 3 |
| 17 | 260818 | 1188 | 1 | 1 | 1 |
| 18 | 269746 | 1188 | 1 | 2 | 2 |
| 19 | 288079 | 1188 | 1 | 1 | 1 |
| 20 | 313897 | 1188 | 1 | 2 | 2 |

接下來若思考這些同學是否真的都花很多時間針對 question_屙＝1188 答題，可將答題時間 (ans_time) 列入 aggregate operator 中取 average



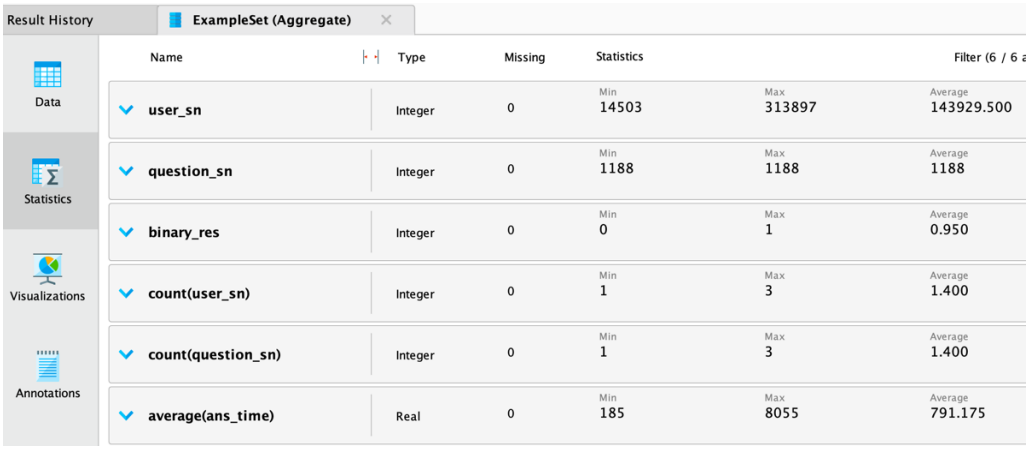


Result History | ExampleSet (Aggregate)

Data | Statistics | Visualizations | Annotations

Open in: Turbo Prep | Auto Model | Interactive Analysis

| Row No. | user_sn | question_sn | binary_res | count(user... | count(ques... | average(an... |
|---|---|---|---|---|---|---|
| 1 | 14503 | 1188 | 1 | 1 | 1 | 185 |
| 2 | 15995 | 1188 | 1 | 1 | 1 | 374 |
| 3 | 32737 | 1188 | 1 | 1 | 1 | 365 |
| 4 | 38614 | 1188 | 1 | 1 | 1 | 632 |
| 5 | 49363 | 1188 | 1 | 2 | 2 | 533 |
| 6 | 54952 | 1188 | 0 | 1 | 1 | 480 |
| 7 | 54952 | 1188 | 1 | 1 | 1 | 480 |
| 8 | 65038 | 1188 | 1 | 1 | 1 | 487 |
| 9 | 106042 | 1188 | 1 | 1 | 1 | 413 |
| 10 | 112627 | 1188 | 1 | 2 | 2 | 8055 |
| 11 | 122416 | 1188 | 1 | 2 | 2 | 836 |
| 12 | 162052 | 1188 | 1 | 1 | 1 | 429 |
| 13 | 203770 | 1188 | 1 | 1 | 1 | 468 |
| 14 | 223594 | 1188 | 1 | 2 | 2 | 304.500 |
| 15 | 243283 | 1188 | 1 | 1 | 1 | 375 |
| 16 | 246112 | 1188 | 1 | 3 | 3 | 271 |
| 17 | 260818 | 1188 | 1 | 1 | 1 | 348 |
| 18 | 269746 | 1188 | 1 | 2 | 2 | 310 |
| 19 | 288079 | 1188 | 1 | 1 | 1 | 204 |
| 20 | 313897 | 1188 | 1 | 2 | 2 | 274 |



Result History | ExampleSet (Aggregate)

Data | Statistics | Visualizations | Annotations

| Name | Type | Missing | Statistics | | |
|---|---|---|---|---|---|
| user_sn | Integer | 0 | Min 14503 | Max 313897 | Average 143929.500 |
| question_sn | Integer | 0 | Min 1188 | Max 1188 | Average 1188 |
| binary_res | Integer | 0 | Min 0 | Max 1 | Average 0.950 |
| count(user_sn) | Integer | 0 | Min 1 | Max 3 | Average 1.400 |
| count(question_sn) | Integer | 0 | Min 1 | Max 3 | Average 1.400 |
| average(ans_time) | Real | 0 | Min 185 | Max 8055 | Average 791.175 |

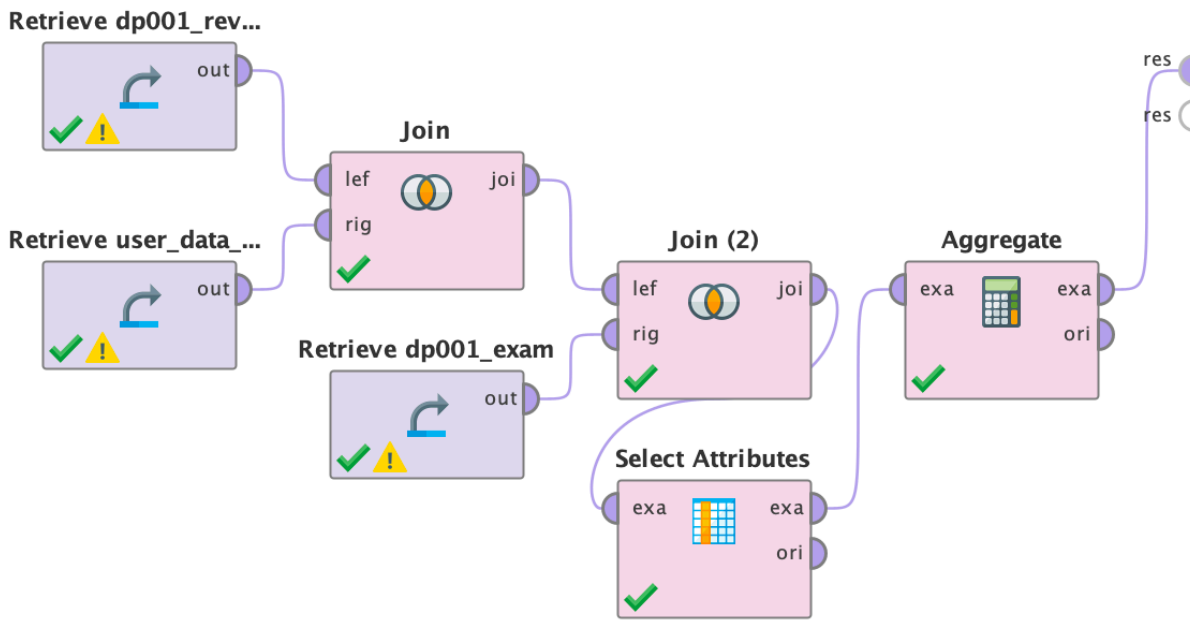
可見最大值為 8055 秒，最小 185 秒，平均要花上 791.175 秒，這 28 次的作答結果，不管答對（19 人）或答錯（1 人），大家的確是花了不少時間作答。

再接下來請思考如何針對 question_sn = 1209 以及 question_sn = 981 這兩個同樣也花了很多時間完成作答的題目試著分析看看。

提示： 將 aggregate operator 中的 aggregate attributes 以及 group by attributes 移除，也就是只處理 binary_res 以及 ans_time 並限定 question_sn = 1209 ，結果可得答錯的同學只有一位花費 1505 秒，其餘 22 位同學平均花費 467.792 秒；而 question_sn = 981 答錯的同學有兩位，平均花費 1068.5 秒，其它 30 位同學平均花費 609.967 秒，你覺得這兩題列在答題秒數前十名的榜單內，原因為何？

# 建模預測: dp001_exam

完成以下查詢，以學生代號進行 group by，計算各生之作答次數、作答正確率、平均作答時間以及其學期成積組別( cluster_0 or cluster_1)



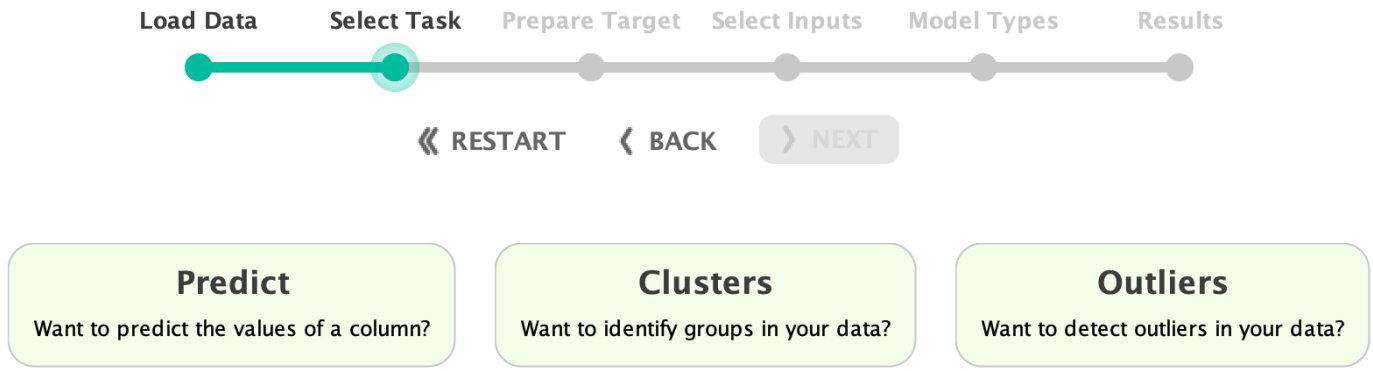

Result History | ExampleSet (Aggregate)

Open in | Turbo Prep | Auto Model | Interactive Analysis

Data | Statistics | Visualizations | Annotations

| Row No. | user_sn | count(question_sn) | average(ans_time) | average(binary_res) | least(cluster) |
|---|---|---|---|---|---|
| 1 | 8634 | 5 | 23.800 | 0.600 | cluster_1 |
| 2 | 11303 | 16 | 135.312 | 0.312 | cluster_1 |
| 3 | 11312 | 32 | 307.406 | 0.844 | cluster_0 |
| 4 | 13328 | 27 | 35.815 | 0.667 | cluster_0 |
| 5 | 14503 | 180 | 388.194 | 0.822 | cluster_1 |
| 6 | 15995 | 166 | 685.771 | 0.867 | cluster_0 |
| 7 | 20034 | 27 | 197.185 | 0.407 | cluster_0 |
| 8 | 23250 | 27 | 33.852 | 0.630 | cluster_0 |
| 9 | 32737 | 166 | 327.024 | 0.867 | cluster_0 |
| 10 | 38371 | 36 | 444.111 | 0.639 | cluster_1 |

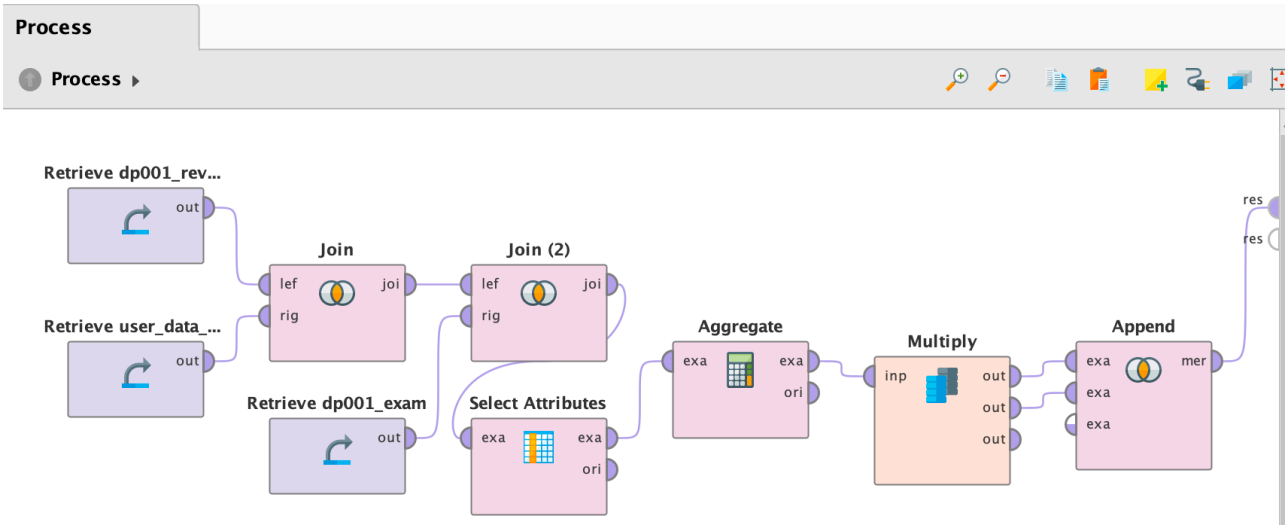
接著在查詢結果中直接點按 Auto Model，由於我們接下來想經由以上收集的維度預測其學期成績組別，故選擇 Predict，並指定 cluster 為 lable



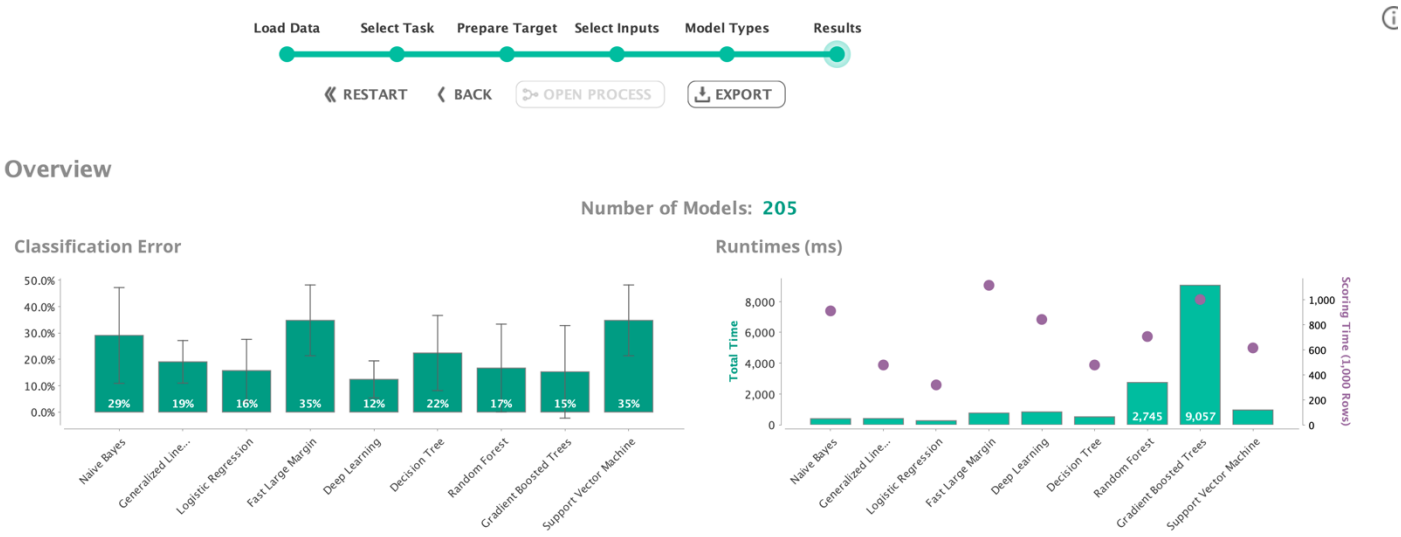

由於受測資料不足 100 筆，無法使用 AutoModel ，解決方式可用 oversampling 或手動執行單一演算法的預測工作，說明如下：

1. Oversampling

使用 multiply operator 以及 append operator 將 55 筆資料增倍



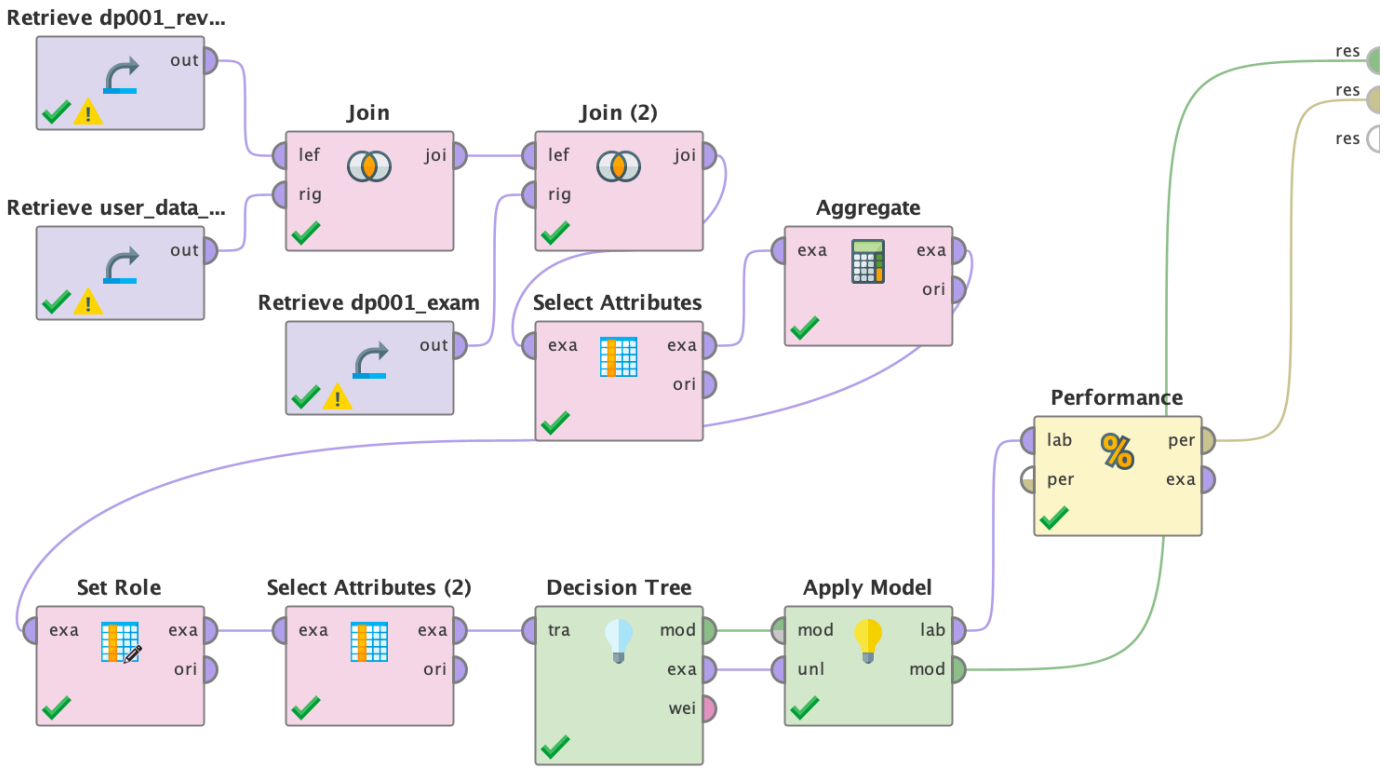

完成後，即可使用 AutoModle，指定 lable 之後即可執行，結果如下：




Classification Error

| Model | Classification Error | Standard Deviation | Gains | Total Time | Training Time (1,000... | Scoring Time (1,000 ... |
|---|---|---|---|---|---|---|
| Naive Bayes | 29.0% | ± 18.2% | 2 | 395 ms | 209 ms | 909 ms |
| Generalized Linear Model | 19.0% | ± 8.1% | 4 | 402 ms | 355 ms | 477 ms |
| Logistic Regression | 15.7% | ± 11.9% | 14 | 264 ms | 218 ms | 318 ms |
| Fast Large Margin | 34.8% | ± 13.4% | 0 | 760 ms | 236 ms | 1 s |
| Deep Learning | 12.4% | ± 7.0% | 16 | 834 ms | 2 s | 841 ms |
| Decision Tree | 22.4% | ± 14.2% | 8 | 518 ms | 218 ms | 477 ms |
| Random Forest | 16.7% | ± 16.7% | 16 | 3 s | 318 ms | 705 ms |
| Gradient Boosted Trees | 15.2% | ± 17.5% | 12 | 9 s | 3 s | 1 s |
| Support Vector Machine | 34.8% | ± 13.4% | 0 | 959 ms | 218 ms | 614 ms |

2. 手動預測



Criterion

- accuracy
- classification error
- kappa

Table View | Plot View

**accuracy: 89.09%**

| | true cluster_1 | true cluster_0 | class precision |
|---|---|---|---|
| pred. cluster_1 | 17 | 5 | 77.27% |
| pred. cluster_0 | 1 | 32 | 96.97% |
| class recall | 94.44% | 86.49% | |

若使用 Performance Binominal Classification 可獲得兩兩類別比較的更多資訊

